# 15

# Hensel lifting and factoring polynomials

In this chapter, we present two modular algorithms for factoring in $\mathbb{Q}[x]$ and $F[x,y]$ for a field $F$. The first one uses factorization modulo a "big" prime and is conceptually easier, and the second one uses factorization modulo a "small" prime and then "lifts" it to a factorization modulo a power of that prime. The latter is computationally faster and comprises our most powerful employment of the prime power modular approach introduced in Chapter 5.

## 15.1. Factoring in $\mathbb{Z}[x]$ and $\mathbb{Q}[x]$: the basic idea

Our first goal is to understand the difference between "factoring in $\mathbb{Z}[x]$" and "factoring a polynomial with integer coefficients in $\mathbb{Q}[x]$". The basic fact is that the latter corresponds to factoring primitive polynomials in $\mathbb{Z}[x]$, while the former requires in addition the factoring of an integer, namely the polynomial's content. We rely on the following notions which were introduced in Section 6.2.

Let $R$ be a Unique Factorization Domain (our two main applications are, as usual, $R = \mathbb{Z}$ and $R = F[y]$ for a field $F$). The **content** $\operatorname{cont}(f)$ of a polynomial $f \in R[x]$ is the greatest common divisor of its coefficients (with the convention that the gcd is positive if $R = \mathbb{Z}$ and monic if $R = F[y]$). The **primitive part** $\operatorname{pp}(f)$ of $f$ is $f/\operatorname{cont}(f) \in R[x]$, and $f$ is **primitive** if $\operatorname{cont}(f) = 1$. Corollary 6.7, a consequence of Gauß' lemma, says that for any $f, g \in R[x]$, we have $\operatorname{cont}(fg) = \operatorname{cont}(f)\operatorname{cont}(g)$ and $\operatorname{pp}(fg) = \operatorname{pp}(f)\operatorname{pp}(g)$. In particular, if $f$ and $g$ are primitive, then so is $fg$. As a consequence, $R[x]$ is a Unique Factorization Domain, and its primes are the primes of $R$ plus the primitive polynomials in $R[x]$ that are irreducible in $K[x]$, where $K$ is the field of fractions of $R$.

Thus the relation between factoring polynomials over $\mathbb{Q}$ and over $\mathbb{Z}$ is as follows. If $f \in \mathbb{Z}[x]$ is primitive, then a factorization $f = f_1 \cdots f_k$ into irreducible $f_i \in \mathbb{Q}[x]$ yields a factorization $f = f_1^* \cdots f_k^*$ into irreducible $f_i^* \in \mathbb{Z}[x]$ by multiplying up denominators and then removing contents. On the other hand, any irreducible factorization in $\mathbb{Z}[x]$ is also one in $\mathbb{Q}[x]$. For an arbitrary $f$ the factorization of $f$ is the factorization of the content of $f$ together with the factorization of the primitive



https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

part of $f$. Thus

$$\text{factoring in } \mathbb{Z}[x] \longleftrightarrow \text{factoring in } \mathbb{Q}[x] \text{ plus factoring in } \mathbb{Z}.$$

The best known algorithms for factoring in $\mathbb{Z}$ (Chapter 19) are much less efficient than those for $\mathbb{Q}[x]$ that we present in this and the next chapter. From now on, "factoring in $\mathbb{Z}[x]$" will usually refer to primitive polynomials, for which the part "factoring in $\mathbb{Z}$" is trivial.

The basic idea of the factoring algorithm is as follows. Let $f \in \mathbb{Z}[x]$ be a primitive polynomial to be factored. Using the squarefree part algorithm 14.19 if necessary, we may assume that $f$ is squarefree. We take a "big" prime $p \in \mathbb{Z}$ not dividing the leading coefficient of $f$ and such that $f \bmod p \in \mathbb{F}_p[x]$ is squarefree (we will make precise later what "big" means). Using one of the (probabilistic) algorithms in Chapter 14, we factor $f$ modulo $p$. If $g \in \mathbb{Z}[x]$ is a factor of $f$ and $g_1, \ldots, g_s$ are the irreducible factors of $g$ modulo $p$, then we can recover $g$ from them. If $f$ factors as $f = f_1 \cdots f_k$ in $\mathbb{Z}[x]$, then also $\overline{f} = \overline{f_1} \cdots \overline{f_k}$ in $\mathbb{Z}_p[x]$, where the bar means taking each coefficient modulo $p$. But if the true factor $f_1$ is irreducible, then $\overline{f_1}$ need not be irreducible, and our factorization modulo $p$ will return all the irreducible modular factors of all $f_i$, but we will not immediately know which of them belong together (see Figure 15.2).

The following questions arise if we want to turn this sketch into an algorithm.

- How large do we have to choose $p$ so that we can recover the coefficients of any factor from its image modulo $p$? The answer has already been given by **Mignotte's bound** 6.33 in Section 6.6.
- In what range do we have to choose a random $p$ so that $f \bmod p$ is squarefree with sufficiently high probability? The answer to this question is provided by the **resultant theory** from Chapter 6 and the **prime number theorem** in Chapter 18.
- Finally, the trickiest question is: how can we find the modular factors of $f \bmod p$ that correspond to a true factor of $f$ in $\mathbb{Z}[x]$? Very easy: we simply try all possible **factor combinations**. Unfortunately, this leads to an exponential algorithm in the worst case; examples are given by the **Swinnerton-Dyer polynomials** in Section 15.3. In Chapter 16, we present a method to circumvent this: **short vectors in lattices**.

Our algorithms consist of two stages: a modular factorization stage, where we take either a single big prime or a prime power as modulus, and a second stage where we try to find true factors from modular ones, either by factor combination or with the aid of short vectors in lattices. This is illustrated in Figure 15.1; each of the two variants in the top row may be freely combined with each of the two methods in the bottom row. The next section describes the "big prime" and "factor combination" stages, Sections 15.4 and 15.5 the "prime power" approach, and "short vectors" are treated in Chapter 16.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

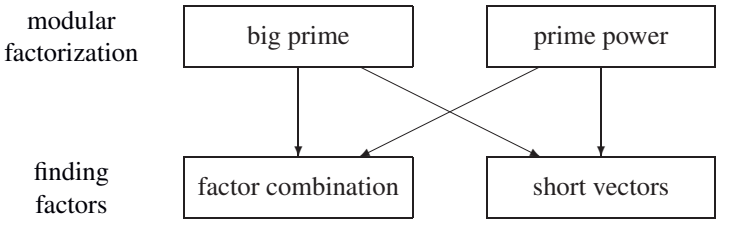



FIGURE 15.1: The building blocks of factoring algorithms for $\mathbb{Z}[x]$.

## 15.2. A factoring algorithm

Let $f \in \mathbb{Z}[x]$ be squarefree. We first have to understand for which primes $p$ the polynomial $\overline{f} = f \bmod p \in \mathbb{Z}_p[x]$ is squarefree, where the bar denotes the reduction modulo $p$. We abbreviate the **discriminant** of $f$ as $\operatorname{disc}(f) = \operatorname{res}(f, f')$, where $'$ is the formal derivative. (In the literature, there are other definitions of the discriminant which differ slightly from ours.) Thus Corollary 14.25 says that $\overline{f}$ is squarefree if and only if $\operatorname{disc}(\overline{f}) \neq 0$. From Lemma 6.25 and the fact that $\overline{f'} = \overline{f}'$ (Exercise 9.23), we obtain the following.

LEMMA 15.1. *Let $f \in \mathbb{Z}[x]$ be a nonzero squarefree polynomial, $p \in \mathbb{N}$ a prime not dividing $\operatorname{lc}(f)$, and denote by a bar the reduction modulo $p$. Then $\overline{f}$ is squarefree if and only if $p \nmid \operatorname{disc}(f)$.*

A closer look at the Sylvester matrix shows that $\operatorname{lc}(f)$ divides $\operatorname{res}(f, f')$ (Exercise 6.41). Hence $\overline{f}$ is squarefree if $p$ does not divide $\operatorname{res}(f, f') \in \mathbb{Z} \setminus \{0\}$; the resultant is nonzero because $f$ is squarefree.

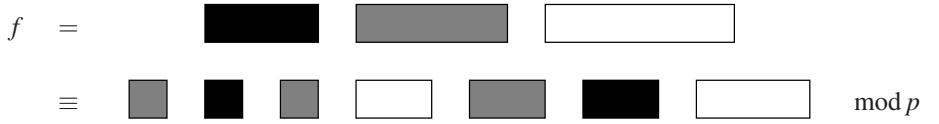



FIGURE 15.2: Factorization patterns in $\mathbb{Z}[x]$ and $\mathbb{F}_p[x]$.

Now let $f = f_1 \cdots f_k$ be the factorization of $f$ into (primitive) irreducible polynomials $f_1, \ldots, f_k$ in $\mathbb{Z}[x]$. Then

$$\overline{f} = \overline{f_1} \cdots \overline{f_k} = \overline{\operatorname{lc}(f)} g_1 \cdots g_r$$

in $\mathbb{F}_p[x]$, where $g_1, \ldots, g_r$ are the monic irreducible factors of $\overline{f}$ in $\mathbb{F}_p[x]$. This is illustrated in Figure 15.2 for $k = 3$ and $r = 7$: the boxes represent irreducible factors, the width of a box corresponds to the degree, and a box in the upper row splits modulo $p$ into the boxes in the lower row of the same color. Suppose that by

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

factoring $f$ modulo $p$, we have computed all the $g_i$. Let $f_1$ be an irreducible factor of $f$ (say, the gray box in the upper row of Figure 15.2). Modulo $p$ some of the $g_i$'s divide $f_1$ (the gray boxes in the lower row). We let $S \subseteq \{1,\dots,r\}$ be the set of indices of the irreducible factors of $\overline{f_1}$ (which we do not know). Then

$$\frac{\mathrm{lc}(f)}{\mathrm{lc}(f_1)} f_1 \equiv \mathrm{lc}(f) \prod_{i \in S} g_i \bmod p. \tag{1}$$

If $p/2$ is larger than the Mignotte bound $(n+1)^{1/2} 2^n |\mathrm{lc}(f)| \cdot \|f\|_\infty$, then the coefficients of $\mathrm{lc}(f) f_1 / \mathrm{lc}(f_1)$ are integers less than $p/2$ in absolute value, by Corollary 6.33, and the polynomials in (1) are equal if we use symmetric representatives between $-(p-1)/2$ and $(p-1)/2$ for the elements of $\mathbb{F}_p$. Therefore we can construct $f_1$ from the $g_i$'s and $S$.

Unfortunately, there seems to be no easy way to find the set $S$: in Figure 15.2, we are only given the boxes in the lower row, but do not know which ones have the same color. Trying all subsets of $\{1,\dots,r\}$ leads to the following algorithm. For compatibility with later algorithms, it has no step numbered 4. We recall the max-norm $\|f\|_\infty = \max_i |f_i|$ and the one-norm $\|f\|_1 = \sum_i |f_i|$ of a polynomial $f = \sum_i f_i x^i \in \mathbb{Z}[x]$.

ALGORITHM 15.2 Factorization in $\mathbb{Z}[x]$ (big prime version).

Input: A squarefree primitive polynomial $f \in \mathbb{Z}[x]$ of degree $n \geq 1$ with $\mathrm{lc}(f) > 0$ and max-norm $\|f\|_\infty = A$.

Output: The irreducible factors $\{f_1,\dots,f_k\} \subseteq \mathbb{Z}[x]$ of $f$.

1. **if** $n = 1$ **then return** $\{f\}$
   $b \longleftarrow \mathrm{lc}(f)$, $\quad B \longleftarrow (n+1)^{1/2} 2^n A b$

2. **repeat**
   choose a random odd prime number $p$ with $2B < p < 4B$
   $\overline{f} \longleftarrow f \bmod p$
   **until** $\gcd(\overline{f}, \overline{f}') = 1$ in $\mathbb{F}_p[x]$

3. { modular factorization }
   compute $g_1,\dots,g_r \in \mathbb{Z}[x]$ of max-norm less than $p/2$ that are nonconstant, monic, and irreducible modulo $p$, such that $f \equiv b g_1 \cdots g_r \bmod p$

5. { initialize the index set $T$ of modular factors still to be treated, the set $G$ of factors found, and the polynomial $f^*$ still to be factored }
   $T \longleftarrow \{1,\dots,r\}$, $\quad s \longleftarrow 1$, $\quad G \longleftarrow \emptyset$, $\quad f^* \longleftarrow f$

6. { factor combination }
   **while** $2s \leq \#T$ **do**

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

7. **for** all subsets $S \subseteq T$ of cardinality $\#S = s$ **do**

8. compute $g^*, h^* \in \mathbb{Z}[x]$ with max-norm less than $p/2$ satisfying $g^* \equiv b\prod_{i\in S} g_i \bmod p$ and $h^* \equiv b\prod_{i\in T\setminus S} g_i \bmod p$

9. **if** $\|g^*\|_1\|h^*\|_1 \le B$ **then**
$T \longleftarrow T \setminus S, \quad G \longleftarrow G \cup \{\mathrm{pp}(g^*)\},$
$f^* \longleftarrow \mathrm{pp}(h^*), \quad b \longleftarrow \mathrm{lc}(f^*)$
break the loop 7 and **goto** 6

10. $s \longleftarrow s+1$

11. **return** $G \cup \{f^*\}$

We recall the multiplication time $\mathsf{M}$ (see inside back cover).

THEOREM 15.3.
*Algorithm 15.2 works correctly. If $\beta = \log B$, then $\beta \in O(n + \log A)$, and the expected cost of steps 2 and 3 is*

$$O\Big(\beta^2\mathsf{M}(\beta)\log\beta + (\mathsf{M}(n^2)+\mathsf{M}(n)\beta)\log n\cdot\mathsf{M}(\beta)\log\beta\Big) \text{ or } O^{\sim}(n^3+\log^3 A)$$

*word operations. One execution of steps 8 and 9 takes*

$$O((\mathsf{M}(n)\log n + n\log\beta)\mathsf{M}(\beta)) \text{ or } O^{\sim}(n^2 + n\log A)$$

*word operations, and there are at most $2^{n+1}$ iterations.*

PROOF. By Lemma 15.1, $f \bmod p$ is squarefree in step 3, since $p > B$ implies that $p \nmid b$.

We show first that the condition in step 9 is true if and only if $g^*h^* = bf^*$. If the latter holds, then $\|g^*\|_1\|h^*\|_1 \le B$, by Corollary 6.33. Conversely, let $g^*$ and $h^*$ be as in step 8. Then $g^*h^* \equiv bf^* \bmod p$. Now $\|g^*h^*\|_\infty \le \|g^*h^*\|_1 \le \|g^*\|_1\|h^*\|_1 \le B < p/2$ implies that both sides of the congruence have coefficients less than $p/2$ in absolute value, and hence they are equal.

For a factor $u \in \mathbb{Z}[x]$ of $f$, we denote by $\mu(u)$ the number of monic irreducible factors which divide $u$ modulo $p$; since $\mathbb{F}_p[x]$ is a UFD, these factors form a subset of $\{g_1,\dots,g_r\}$. We show by induction that the invariants

$$\begin{aligned} &f^* \equiv b\textstyle\prod_{i\in T} g_i \bmod p, \quad b = \mathrm{lc}(f^*), \quad f = f^*\prod_{g\in G} g, \\ &\text{each polynomial in } G \text{ is irreducible,} \\ &f^* \text{ is primitive and each of its irreducible factors } u \in \mathbb{Z}[x] \text{ has } \mu(u) \ge s \end{aligned} \tag{2}$$

hold each time the algorithm passes through step 6. This is clear initially. So we assume that the invariants hold before step 8, and that the condition in step 9

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

is true for some subset $S \subseteq T$ of cardinality $s$. Then $g^*h^* = bf^*$, by the above, and by Gauß' lemma, $\mathrm{pp}(g^*)$ is a factor of $\mathrm{pp}(bf^*) = f^*$. Since $\mu(g^*) = s$ and $\mu(u) \geq s$ for each irreducible factor $u$ of $f^*$, $\mathrm{pp}(g^*)$ is an irreducible factor of $f$. The actions taken in step 9 then ensure that the invariants hold the next time through step 6. Now we assume that the condition in step 9 is false for all subsets $S \subseteq T$ of cardinality $s$, but that $f^*$ has an irreducible factor $g \in \mathbb{Z}[x]$ with $\mu(g) = s$. Then there is a subset $S \subseteq T$ of cardinality $s$ such that $g \equiv \mathrm{lc}(g) \prod_{i \in S} g_i \bmod p$, since $\mathbb{F}_p[x]$ is a UFD. We set $h = f^*/g \in \mathbb{Z}[x]$. Then $\mathrm{lc}(g)\,\mathrm{lc}(h) = \mathrm{lc}(f^*) = b$,

$$\mathrm{lc}(h)g \equiv b \prod_{i \in S} g_i \equiv g^* \bmod p, \text{ and } \mathrm{lc}(g)h \equiv b \prod_{i \in T \setminus S} g_i \equiv h^* \bmod p.$$

By Mignotte's bound 6.33, the coefficients of $\mathrm{lc}(h)g$ and $\mathrm{lc}(g)h$ are at most $B < p/2$ in absolute value. Since also $\|g^*\|_\infty, \|h^*\|_\infty < p/2$, we have $\mathrm{lc}(h)g = g^*$, $\mathrm{lc}(g)h = h^*$, $g^*h^* = bf^*$, and the condition in step 9 is true for that particular subset $S$ of $T$. This contradiction shows that $f^*$ has no irreducible factor $g$ with $\mu(g) = s$, and step 10 guarantees that the invariants hold again at the next pass through step 6.

It remains to show that $f^*$ is irreducible if $2s > \#T$ in step 6. Let $g \in \mathbb{Z}[x]$ be an irreducible factor of $f^*$ and $h = f^*/g$. By (2), we have $s \leq \mu(g), \mu(h) \leq \#T$ if $h$ is nonconstant. But $\mu(g) + \mu(h) = \#T$, and $s > \#T/2$ implies that $h = \pm 1$ and $f^* = \pm g$ is irreducible.

For the running time estimate, we first note that $b \leq \|f\|_\infty = A$, and hence $\beta \in O(n + \log A)$. In Section 18.4, we show that a random prime as required in step 2 can be found by a probabilistic algorithm using $O(\beta^2 \mathsf{M}(\beta) \log \beta)$ word operations, and that $p \nmid \mathrm{disc}(f)$ with probability at least $1/2$ (Corollary 18.12). Hence the expected number of iterations of step 2 is at most two. The cost for the gcd is $O(\mathsf{M}(n) \log n)$ arithmetic operations in $\mathbb{F}_p$ or $O(\mathsf{M}(n) \log n\, \mathsf{M}(\beta) \log \beta)$ word operations. Thus the expected cost of step 2 is $O((\beta^2 + \mathsf{M}(n) \log n)\mathsf{M}(\beta) \log \beta)$ word operations. By Corollary 14.30, step 3 can be done with $O((\mathsf{M}(n^2) + \mathsf{M}(n)\beta) \log n)$ arithmetic operations in $\mathbb{F}_p$. Each of these in turn takes $O(\mathsf{M}(\beta) \log \beta)$ word operations, by Corollary 11.13, and the expected number of word operations for step 3 is $O((\mathsf{M}(n^2) + \mathsf{M}(n)\beta) \log n \cdot \mathsf{M}(\beta) \log \beta)$.

Computing $g^*$ and $h^*$ in step 8 can be done with $O(\mathsf{M}(n) \log n)$ additions and multiplications modulo $p$, by Lemma 10.4, or $O(\mathsf{M}(n) \log n\, \mathsf{M}(\beta))$ word operations. The primitive parts in step 9 can be computed with at most $n$ gcds of integers absolutely bounded by $B$, or $O(n\, \mathsf{M}(\beta) \log \beta)$ word operations. This has to be done $k \leq n$ many times. Between two subsequent times that the condition in step 9 is true, there are at most $2^{\#T}$ executions of step 8. Now $\#T$ decreases by at least one if the condition is true, and hence the total number of iterations of step 8 is at most

$$\sum_{1 \leq i \leq r} 2^i \leq 2^{r+1} \leq 2^{n+1}. \quad \square$$

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

EXAMPLE 15.4. Let $f = 6x^4 + 5x^3 + 15x^2 + 5x + 4 \in \mathbb{Z}[x]$. Then $f$ is primitive, $n = 4$, $A = 15$, $b = \mathrm{lc}(f) = 6$, and we have $f' = 24x^3 + 15x^2 + 30x + 5$ and $\mathrm{disc}(f) = 19250814$, so that $f$ is squarefree, and $B = (n+1)^{1/2}2^n Ab = 1440\sqrt{5} \approx 3219.9$. Suppose that we choose $p = 6473 > 2B$ in step 2. Then $p \nmid b$, and $f$ and $f'$ are coprime modulo $p$. In fact, the only "unlucky" primes for $f$, with $p \mid \mathrm{lc}(f)$ or $f \bmod p$ not squarefree, are the prime divisors $2, 3, 11, 31$, and $97$ of $\mathrm{disc}(f)$.

In step 3, we obtain the modular factorization

$$f \equiv bg_1g_2g_3g_4 = 6(x-819)(x+605)(x+2632)(x+2977) \bmod p.$$

We now trace steps 8 and 9 for two specific subsets $S \subseteq \{1, \ldots, 4\}$. It turns out that the condition in step 9 is false for all subsets $S \subseteq \{1, \ldots 4\}$ of cardinality $s = 1$, and $f^* = f$ has no linear factor. For $s = 2$ and $S = \{1, 2\}$, we compute

$$\begin{aligned} g^* &\equiv bg_1g_2 = 6(x-819)(x+605) \equiv 6x^2 - 1284x - 1863 \bmod p, \\ h^* &\equiv bg_3g_4 = 6(x+2632)(x+2977) \equiv 6x^2 + 1289x - 615 \bmod p \end{aligned}$$

in step 8. Obviously $\|g^*\|_1\|h^*\|_1 \geq \|g^*\|_\infty\|h^*\|_\infty = 1863 \cdot 1289 > B$ in step 9, and in fact $g^*h^* \neq bf^*$, which can be seen by comparing the constant coefficients

$$g^*(0)h^*(0) = (-1863) \cdot (-615) \neq 24 = 6 \cdot 4 = bf^*(0).$$

On the other hand, if we take $S = \{1, 4\}$, then

$$\begin{aligned} g^* &\equiv bg_1g_4 = 6(x-819)(x+2977) \equiv 6x^2 + 2x + 2 \bmod p, \\ h^* &\equiv bg_2g_3 = 6(x+605)(x+2632) \equiv 6x^2 + 3x + 12 \bmod p \end{aligned}$$

in step 8, $\|g^*\|_1\|h^*\|_1 = 10 \cdot 21 < B$ in step 9, and in fact $g^*h^* = bf^*$, so that $\mathrm{pp}(g^*) = 3x^2 + x + 1$ and $\mathrm{pp}(h^*) = 2x^2 + x + 4$ are the irreducible factors of $f$ in $\mathbb{Z}[x]$. $\diamond$

Before computing $g^*$ and $h^*$ in step 8, one will test first whether the constant coefficients of $g^*h^*$ and $bf^*$ are equal (unless $f(0) = 0$, which can be ruled out in advance), as in Example 15.4. They can be computed with at most $r$ multiplications of integers of absolute value at most $B$ or $O(r \cdot \mathsf{M}(n + \log A))$ word operations, which is much faster than the worst case bound for steps 8 and 9 in the theorem. In practice, most unsuccessful $g^*$ and $h^*$ already fail this simple test. Instead of multiplying with $b$, we might also compute the monic associates of $g^*$ and $h^*$ by rational number reconstruction (Section 5.10). If the constant coefficient of $f$ is smaller than $b$, then exchanging the roles of the leading and the constant coefficient decreases the required size of $p$. These remarks also apply to the prime power algorithms 15.19 and 15.22 below and the corresponding algorithms in Chapter 16.

To factor an arbitrary polynomial $f \in \mathbb{Z}[x]$, we might apply Algorithm 15.2 to the squarefree part $h/\gcd(h, h') \in \mathbb{Z}[x]$ of $h = \mathrm{pp}(f)$ and afterwards determine the

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

multiplicities. We use an alternative approach, where we apply it to the polynomials in the squarefree decomposition of $h$ (Section 14.6). Here is the algorithm for factoring in $\mathbb{Q}[x]$ arbitrary polynomials with integer coefficients.

ALGORITHM 15.5 Polynomial factorization in $\mathbb{Q}[x]$.
Input: A polynomial $f \in \mathbb{Z}[x]$ with $\deg f = n \geq 1$ and max-norm $\|f\|_\infty = A$.
Output: A constant $c \in \mathbb{Z}$ and a set of pairs $\{(f_1, e_1), \ldots, (f_k, e_k)\}$ such that the $f_i \in \mathbb{Z}[x]$ are irreducible and pairwise coprime, the $e_i \in \mathbb{N}$ are positive, and $f = c f_1^{e_1} \cdots f_r^{e_r}$.

1. $c \longleftarrow \operatorname{cont}(f), \quad g \longleftarrow \operatorname{pp}(f)$
   **if** $\operatorname{lc}(f) < 0$ **then** $c \longleftarrow -c, \quad g \longleftarrow -g$

2. **call** Yun's algorithm 14.21 to compute the squarefree decomposition $g = \prod_{1 \leq i \leq s} g_i^i$ of $g$, with squarefree and pairwise coprime polynomials $g_1, \ldots, g_s$ in $\mathbb{Z}[x]$ such that $\operatorname{lc}(g_i) > 0$ for all $i$ and $g_s$ is nonconstant.

3. $G \longleftarrow \emptyset$
   **for** $i = 1, \ldots, s$ **do**

4. **call** Algorithm 15.2 to compute all irreducible factors $h_1, \ldots, h_t \in \mathbb{Z}[x]$ of $g_i$
   $G \longleftarrow G \cup \{(h_1, i), \ldots, (h_t, i)\}$

5. **return** $c$ and $G$

If we use Algorithm 14.21 literally in step 2, then it computes monic polynomials with rational coefficients. We obtain the squarefree decomposition into primitive polynomials in $\mathbb{Z}[x]$ by multiplying with a common denominator, or by performing all gcd computations in Algorithm 14.21 in $\mathbb{Z}[x]$ instead of $\mathbb{Q}[x]$, or by using a modular approach (Exercises 15.26 and 15.27).

We forego a cost analysis, because Algorithms 15.2 and 15.5 have exponential running time in the worst case. This does not follow from the upper bound of Theorem 15.3, but we will see in Section 15.3 that there are polynomials for which the algorithm takes exponentially many steps. In Chapter 16, we will present and analyze a polynomial-time algorithm for factoring in $\mathbb{Q}[x]$.

Algorithm 15.2 can be easily adapted to bivariate polynomials over a field $F$, provided that we know how to factor univariate polynomials over algebraic extensions of $F$. We have solved this problem over finite fields, but the prime power algorithm in Section 15.6 below only requires univariate factorization over $F$ itself, and thus also applies to $F = \mathbb{Q}$. For this reason, we forego the details of the adaptation mentioned above.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

## 15.3. Frobenius' and Chebotarev's density theorems

We briefly explain how the factorization of $f \in \mathbb{Z}[x]$ is related to factorizations of $f \bmod p \in \mathbb{F}_p[x]$ for various primes $p \in \mathbb{Z}$. This section requires some familiarity with basic Galois theory. It is not required later and may be skipped at first reading.

We may assume that $f$ is primitive and $p \nmid \operatorname{lc}(f)$, and let $f = f_1 \cdots f_k$ be its factorization, with $f_1, \ldots, f_k \in \mathbb{Z}[x]$ irreducible. Then of course

$$f \bmod p = (f_1 \bmod p) \cdots (f_k \bmod p)$$

in $\mathbb{F}_p[x]$. The computational difficulty is that some $f_i \bmod p$ may not be irreducible in $\mathbb{F}_p[x]$, and that the factor combination stage may have to try exponentially many combinations of the irreducible factors that were computed modulo $p$.

An example of a "bad" polynomial is the $i$th **Swinnerton-Dyer polynomial**

$$f = \prod (x \pm \sqrt{2} \pm \sqrt{3} \pm \sqrt{5} \pm \cdots \pm \sqrt{p_i}) \in \mathbb{Z}[x],$$

where $p_i$ is the $i$th prime and the product runs over all $2^i$ possible combinations of $+$ and $-$ signs. It follows from Galois theory that $f$ is an irreducible polynomial of degree $2^i$ in $\mathbb{Z}[x]$. But since for any prime $p$, $\mathbb{F}_{p^2}$ contains all the square roots $\sqrt{2} \bmod p, \ldots, \sqrt{p_i} \bmod p$, the reduction of $f$ modulo $p$ splits into linear factors over $\mathbb{F}_{p^2}$. Hence the irreducible factors of $f \bmod p$ in $\mathbb{F}_p[x]$ are either all linear (namely, if $2, 3, \ldots, p_i$ are squares modulo $p$) or all quadratic, if $p$ does not divide the discriminant of $f$ (Exercise 15.8), and there are at least $2^{i-1} = n/2$ of them, where $n = 2^i$ is the degree of $f$. Then the factorization algorithm 15.2 will run through about $2^{n/4}$ sets $S$ before it is finally able to decide that $f$ is irreducible. Other examples of "bad" polynomials are the cyclotomic polynomials $\Phi_n$, which are irreducible over $\mathbb{Q}$ but split modulo each prime for most $n$ (Exercise 15.7).

The Swinnerton-Dyer polynomials and the cyclotomic polynomials make the factor combination stage work really hard. But is that typical? For example, we have used the fact that squarefree polynomials "usually" remain squarefree modulo a prime. Can we hope that "usually" an irreducible polynomial in $\mathbb{Z}[x]$ remains irreducible modulo a prime? The answer is no, and the powerful theorems of Frobenius (1896) and Chebotarev (1926) give precise information. Their explanation requires some concepts not used elsewhere in this text; the background and a proof can be found in Stevenhagen & Lenstra (1996).

So we have a primitive polynomial $f \in \mathbb{Z}[x]$ of degree $n$, irreducible over $\mathbb{Q}$. We let $G$ be the Galois group of the splitting field of $f$ over $\mathbb{Q}$. Each automorphism in $G$ is a permutation of the $n$ roots of $f$ and has a unique decomposition into disjoint cycles, say of lengths $\lambda_1, \ldots, \lambda_r$. Then $\lambda_1 + \cdots + \lambda_r = n$, so that $\lambda = (\lambda_1, \ldots, \lambda_r)$ is a partition of $n$. For an arbitrary partition $\lambda$ of $n$, we let $H_\lambda \subseteq G$ be the set of those automorphisms that have cycle decomposition $\lambda$. Thus $\mu(\lambda) = \#H_\lambda / \#G$ is the relative frequency with which the cycle type $\lambda$ occurs in $G$.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

EXAMPLE 15.6. We consider the three irreducible polynomials of degree $n = 4$

$$f_1 = x^4 - 6x^3 - 5x^2 + 8,$$
$$f_2 = x^4 + x^3 + x^2 + x + 1,$$
$$f_3 = x^4 - 10x^2 + 1 = \prod(x \pm \sqrt{2} \pm \sqrt{3})$$

in $\mathbb{Q}[x]$. The first one $f_1$ was chosen at random from the monic polynomials in $\mathbb{Z}[x]$ with coefficients absolutely less than 10, $f_2 = \Phi_5$ is the 5th cyclotomic polynomial, and $f_3$ is a Swinnerton-Dyer polynomial. With appropriate numberings of the roots of $f_1, f_2, f_3$, their Galois groups are $\mathrm{Gal}(f_1) = S_4$, the full symmetric group on four letters, $\mathrm{Gal}(f_2) = \langle (1234) \rangle \cong \mathbb{Z}_4$, the cyclic group with four elements, and $\mathrm{Gal}(f_3)$ is **Klein's group** $V_4 \cong \mathbb{Z}_2 \times \mathbb{Z}_2$. The partitions of 4, that is, the possible cycle types of automorphisms of $f_1, f_2, f_3$, are $(1,1,1,1)$, $(2,1,1)$, $(2,2)$, $(3,1)$, and $(4)$. For each of the three polynomials and each cycle type $\lambda$, Table 15.3 lists the automorphisms of type $\lambda$ in the Galois group. The fractions in bold are the relative frequencies. $\diamond$

| cycle type | $f_1$ | | $f_2$ | | $f_3$ | |
|---|---|---|---|---|---|---|
| $(1,1,1,1)$ | id | $\mathbf{\frac{1}{24}}$ | id | $\mathbf{\frac{1}{4}}$ | id | $\mathbf{\frac{1}{4}}$ |
| $(2,1,1)$ | (12), (13), (14), (23), (24), (34) | $\mathbf{\frac{6}{24}}$ | | | | |
| $(2,2)$ | (12)(34), (13)(24), (14)(23) | $\mathbf{\frac{3}{24}}$ | (13)(24) | $\mathbf{\frac{1}{4}}$ | (12)(34), (13)(24), (14)(23) | $\mathbf{\frac{3}{4}}$ |
| $(3,1)$ | (123), (124), (132), (134), (142), (143), (234), (243) | $\mathbf{\frac{8}{24}}$ | | | | |
| $(4)$ | (1234), (1243), (1324), (1342), (1423), (1432) | $\mathbf{\frac{6}{24}}$ | (1234), (1432) | $\mathbf{\frac{2}{4}}$ | | |

TABLE 15.3: Cycle types of the Galois groups of $f_1, f_2, f_3$ and their relative frequencies.

If we factor $f$ modulo a prime $p$ that does not divide $\mathrm{res}(f, f')$, then the degrees $\lambda_1, \ldots, \lambda_r$ of the irreducible factors also form a partition of $n$, the **factorization pattern** $\lambda = (\lambda_1, \ldots, \lambda_r)$ of $f$ modulo $p$. For any partition $\lambda$, we can consider the set $P_\lambda$ of those primes where $\lambda$ is this factorization pattern. Then **Frobenius' density theorem** says that $P_\lambda$ has density $\mu(\lambda)$, so that a randomly chosen prime is in $P_\lambda$ with probability $\mu(\lambda)$. Chebotarev proved a stronger version of this result, and **Chebotarev's density theorem** has become much better known than Frobenius' theorem. For practical purposes, we would like to use this kind of estimate for the primes in our algorithms. Unfortunately, nothing much can be proved about this, because even the best versions of Chebotarev's theorem

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

(Lagarias & Odlyzko 1977, Oesterlé 1979) do not allow us to conclude that the asymptotic density of $P_\lambda$ already applies to the fairly small values of $p$ that we use (or of any size that leads to practical algorithms). However, case studies like Example 15.6 give rise to the hope that more may be true than what can be proved today.

| pattern | $f_1$ | $f_2$ | $f_3$ |
|---|---|---|---|
| $(1,1,1,1)$ | 3.96% | 24.84% | 24.78% |
| $(2,1,1)$ | 25.30% | | |
| $(2,2)$ | 12.70% | 24.91% | 75.22% |
| $(3,1)$ | 33.18% | | |
| $(4)$ | 24.86% | 50.25% | |

TABLE 15.4: Factorization patterns of $f_1, f_2, f_3$ modulo the first 10000 primes not dividing the discriminant.

EXAMPLE 15.6 (continued). For each of the three polynomials $f_1, f_2, f_3$ from Example 15.6, Table 15.4 shows the frequencies of factorization patterns modulo the first 10000 primes where the polynomial is squarefree. One can see that these approximate the relative frequencies of the conjugacy classes quite well. For example, the partition $(2,2)$, corresponding to a factorization into two irreducible quadratic factors, occurs in 12.70% of all cases for $f_1$, which is close to the frequency 12.5% of the cycle type $(2,2)$ in $\mathrm{Gal}(f_1)$. For $f_2$, this factorization pattern occurs about twice as often, in 24.91% of all cases, and 25% of the four elements of its Galois group have cycle type $(2,2)$. $\diamond$

Now $f$ is irreducible modulo $p$ if and only if $p \in P_{(n)}$. For a Swinnerton-Dyer polynomial of degree $n = 2^i$, the Galois group is isomorphic to $\mathbb{Z}_2^i$, and $\mu((n)) = 0$. If $G = S_n$ (which is the case for almost all random polynomials of degree $n$), then $\mu((n)) = 1/n$, so that $f$ remains irreducible modulo $p$ for a fraction of only about $1/n$ of all primes. Moreover, the average number of cycles of a random permutation in $S_n$ is $O(\log n)$ (see Notes 15.3), and Chebotarev's theorem implies that this is also the average number of factors of $f$ modulo a random prime $p$.

The reader who has absorbed the technique of modular algorithms may have wondered why we do not use several small primes and Chinese remaindering to factor polynomials. The reason is that the irreducible factors in $\mathbb{Z}[x]$ split independently modulo different primes, obeying Frobenius' rule, and it would be even harder to fit them together than combining factors modulo a single prime. As a more practical approach, we might factor modulo several primes, choose one of them as the $p$ in the factor combination step, and for each possible combination $S \subseteq \{1,\ldots,r\}$ check whether the degree $\deg \prod_{i\in S} g_i$ is compatible with all other modular factorizations.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

EXAMPLE 15.7. Suppose that $f \in \mathbb{Z}[x]$ of degree 8 is squarefree modulo $2,3,5$ and has the factorization patterns $(3,3,2)$ modulo 2, $(5,2,1)$ modulo 3, and $(4,3,1)$ modulo 5. From the factorization modulo 2, we can conclude that the possible degrees of factors of $f$ in $\mathbb{Z}[x]$ are $D_2 = \{2,3,5,6,8\}$. Similarly, from the factorizations modulo 3 and 5, we obtain $D_3 = \{1,2,3,5,6,7,8\}$ and $D_5 = \{1,3,4,5,7,8\}$. Taking the intersection $D_2 \cap D_3 \cap D_5 = \{3,5,8\}$, we conclude that every nonconstant factor of $f \in \mathbb{Z}[x]$ has degree $3,5$, or 8, so that $f$ is either irreducible or it splits into two irreducible factors of degree 3 and 5. $\diamond$

## 15.4. Hensel lifting

In Chapter 9, we employed Newton iteration for computing inverses and solving polynomial equations modulo $p^l$ over a ring $R$, where an initial solution modulo $p \in R$ is given. Hensel (1918) invented a similar method to compute factorizations of polynomials modulo $p^l$, which Zassenhaus (1969) introduced into modern factorization algorithms, and which—no surprise—had been known to Gauß.

This is the third of the modular algorithms mentioned at the beginning of Chapter 5. In the big prime version, we took a large enough prime, and in the small primes version, we took small prime numbers $m_1, \ldots, m_l$ so that $m_1 \cdots m_l$ is large enough. Now we take a single small prime $p$ and an integer $l$ so that $p^l$ is large enough.

We start with the simplest idea and then add more technical details. Let $R$ be a ring (commutative, with 1, as usual, our two main examples being $R = \mathbb{Z}$ and $R = F[y]$ for a field $F$), $f,g,h \in R[x]$, and $m \in R$ such that $f \equiv gh \bmod m$. We want to "lift" this to a factorization $f \equiv \hat{g}\hat{h} \bmod m^2$. We assume that we have $s,t \in R[x]$ with $sg+th \equiv 1 \bmod m$, and think of $g$ and $h$ as being coprime modulo $m$. When $R/\langle m \rangle$ is a field, we can find these via the Extended Euclidean Algorithm in $(R/\langle m \rangle)[x]$. We now calculate

$$e = f - gh, \quad \hat{g} = g + te, \quad \hat{h} = h + se, \tag{3}$$

and find

$$\begin{aligned} f - \hat{g}\hat{h} &= f - gh - gse - hte - ste^2 = f - gh - (sg+th)e - ste^2 \\ &= (1 - sg - th)e - ste^2 \equiv 0 \bmod m^2, \end{aligned}$$

since $e \equiv 0 \bmod m$ and $1 - sg - th \equiv 0 \bmod m$. Hence $f \equiv \hat{g}\hat{h} \bmod m^2$, so that $\hat{g}\hat{h}$ is a factorization of $f$ modulo $m^2$. Starting with a prime $p$ for $m$ and proceeding inductively (and simultaneously lifting the congruence $sg+th \equiv 1$), we can lift the factorization modulo arbitrary powers of $p$.

EXAMPLE 15.8. In Example 9.24, we computed a nontrivial solution to $x^4 - 1 \equiv 0 \bmod 625$ using Newton iteration with the starting value $x = 2$ modulo 5. This can

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

also be regarded as lifting the factorization

$$x^4 - 1 \equiv (x-2)(x^3 + 2x^2 - x - 2) \bmod 5$$

to a factorization modulo 625. In the above setting, we then have $f = x^4 - 1$, $p = 5$, $g = x^3 + 2x^2 - x - 2$, and $h = x - 2$. The polynomials $g$ and $h$ are coprime modulo 5, and with the Extended Euclidean Algorithm in $\mathbb{F}_5[x]$, we get $s = -2$ and $t = 2x^2 - 2x - 1$ such that $sg + th \equiv 1 \bmod 5$. With $m = p$, we obtain

$$\begin{aligned} e &= f - gh = 5x^2 - 5, \\ \hat{g} &= g + te = 10x^4 - 9x^3 - 13x^2 + 9x + 3, \\ \hat{h} &= h + se = -10x^2 + x + 8, \end{aligned}$$

and in fact

$$f - \hat{g}\hat{h} = 25 \cdot (4x^6 - 4x^5 - 8x^4 + 7x^3 + 5x^2 - 3x - 1) \equiv 0 \bmod 25,$$

so that $f \equiv \hat{g}\hat{h} \bmod 25$. $\diamond$

The above example shows one drawback to our first approach: the degrees of $\hat{g}$ and $\hat{h}$ are higher than those of $g$ and $h$, in particular their sum exceeds the degree of $f$. This may happen because the multiples of $m$ are zero divisors modulo $m^2$, and hence the product of the leading coefficients of two polynomials may vanish modulo $m^2$.

To overcome this problem, we use division with remainder in $R[x]$. Since $R$ is not a field, this is not always possible. The following lemma states that division with remainder by monic polynomials always works.

LEMMA 15.9. *(i) Let $f, g \in R[x]$, with $g$ nonzero and monic. Then there exist unique polynomials $q, r \in R[x]$ with $f = qg + r$ and $\deg r < \deg g$.*

*(ii) If $f, g, q, r$ are as in (i) and $f \equiv 0 \bmod m$ for some $m \in R$, then $q \equiv r \equiv 0 \bmod m$.*

Part (i) has been proven in Section 2.4, and the proof of (ii) is Exercise 15.12. We do not need the coefficients of the new polynomials exactly, but only modulo $m^2$. This means that over a Euclidean domain $R$ we can reduce them accordingly and keep their sizes small. Here are the formulas that work.

ALGORITHM 15.10 Hensel step.

Input: An element $m$ in a (commutative) ring $R$, and polynomials $f, g, h, s, t \in R[x]$ such that

$$f \equiv gh \bmod m \text{ and } sg + th \equiv 1 \bmod m,$$

$\mathrm{lc}(f)$ is not a zero divisor modulo $m$, $h$ is monic, $\deg f = n = \deg g + \deg h$, $\deg s < \deg h$, and $\deg t < \deg g$.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

Output: Polynomials $g^*, h^*, s^*, t^* \in R[x]$ such that

$$f \equiv g^* h^* \bmod m^2 \text{ and } s^* g^* + t^* h^* \equiv 1 \bmod m^2,$$

$h^*$ is monic, $g^* \equiv g \bmod m$, $h^* \equiv h \bmod m$, $s^* \equiv s \bmod m$, and $t^* \equiv t \bmod m$, $\deg g^* = \deg g$, $\deg h^* = \deg h$, $\deg s^* < \deg h^*$, and $\deg t^* < \deg g^*$.

1. compute $e, q, r, g^*, h^* \in R[x]$ such that $\deg r < \deg h$ and

$$\begin{array}{rclcrcl} e & \equiv & f - gh \bmod m^2, & & se & \equiv & qh + r \bmod m^2, \\ g^* & \equiv & g + te + qg \bmod m^2, & & h^* & \equiv & h + r \bmod m^2 \end{array} \tag{4}$$

2. compute $b, c, d, s^*, t^* \in R[x]$ such that $\deg d < \deg h^*$ and

$$\begin{array}{rclcrcl} b & \equiv & sg^* + th^* - 1 \bmod m^2, & & sb & \equiv & ch^* + d \bmod m^2, \\ s^* & \equiv & s - d \bmod m^2, & & t^* & \equiv & t - tb - cg^* \bmod m^2 \end{array} \tag{5}$$

3. **return** $g^*, h^*, s^*, t^*$

EXAMPLE 15.8 (continued). We let $m = 5$ and $f, g, h, s, t, e$ in $\mathbb{Z}[x]$ be as in Example 15.8. Division of $se$ by $h$ with remainder yields $q = -10x + 5$ and $r = -5$ with $se \equiv qh + r \bmod 25$. Moreover, we compute

$$g^* \equiv g + te + qg \equiv x^3 + 7x^2 - x - 7 \bmod 25, \quad h^* \equiv h + r \equiv x - 7 \bmod 25.$$

Then $f \equiv g^* h^* \bmod 25$, and the degrees of $g^*, h^*$ are the same as those of $g$ and $h$; the polynomials are simpler than $\hat{g}, \hat{h}$ as calculated before. As in Example 9.24, we obtain that 7 is a solution to $x^4 - 1 \equiv 0 \bmod 25$ that is congruent to the starting solution 2 modulo 5.

To obtain $s^*, t^*$, which we need for the next iteration, we compute

$$b \equiv sg^* + th^* - 1 \equiv -5x^2 - 10x - 5 \bmod 25.$$

Polynomial division yields $c = 10x - 10$ and $d = -10$ with $sb \equiv ch^* + d \bmod 25$. Now

$$s^* \equiv s - d \equiv 8 \bmod 25, \quad t^* \equiv t - tb - cg^* \equiv -8x^2 - 12x - 1 \bmod 25.$$

Then indeed $s^* g^* + t^* h^* \equiv 1 \bmod 25$, and the degrees of $s^*, t^*$ agree with those of $s, t$, respectively. $\diamond$

THEOREM 15.11.

*Algorithm 15.10 works correctly as specified. It uses $O(\mathsf{M}(n)\mathsf{M}(\log m))$ word operations if $R = \mathbb{Z}$, $m > 1$, and all inputs have max-norm less than $m^2$, and $O(\mathsf{M}(n)\mathsf{M}(\deg_y m))$ operations in the field $F$ if $R = F[y]$ and the degree in $y$ of all inputs is less than $2 \deg_y m$.*

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

PROOF. For the correctness, we only prove the claims about $g^*, h^*$; those for $s^*, t^*$ are left as Exercise 15.17. We calculate

$$\begin{aligned} f - g^*h^* &\equiv f - (g + te + qg)(h + se - qh) \\ &= f - gh - (sg + th)e - ste^2 - (sg - th)qe + ghq^2 \\ &\equiv (1 - sg - th)e - ste^2 - (sg - th)qe + ghq^2 \equiv 0 \bmod m^2, \end{aligned}$$

since we have $1 - sg - th \equiv e \equiv 0 \bmod m$, by assumption, and $q \equiv 0 \bmod m$, by Lemma 15.9. These also imply that $g^* \equiv g \bmod m$ and $h^* \equiv h \bmod m$. Now $\deg r < \deg h$ by construction, and thus $h^*$ is also monic and of the same degree as $h$. Finally, $f \equiv g^*h^* \bmod m^2$ and $h^*$ monic together with the assumption that $\mathrm{lc}(f)$ is not a zero divisor modulo $m$ imply that $\deg g^* = \deg f - \deg h^* = \deg f - \deg h = \deg g$.

We note that for any arithmetic operation (addition, multiplication, division with remainder) of polynomials in $\mathbb{Z}[x]$ occurring in the algorithm, their degrees are at most $2n$, and their coefficients are less than $m^4$, with length in $O(\log m)$. Thus the cost for one arithmetical operation in $\mathbb{Z}$ is $O(\mathsf{M}(\log m))$ word operations. The number of arithmetic operations in $\mathbb{Z}$ for one polynomial addition, multiplication, or division with remainder by a monic polynomial is $O(\mathsf{M}(n))$. The estimate for $R = F[y]$ goes analogously. □

THEOREM 15.12 *Hensel's lemma.*
*Given a nonzero $p \in R$, $l \in \mathbb{N}_{>0}$, and assuming the input specification of Algorithm 15.10 for $m = p$, we can compute polynomials as in the output specification, but with $m^2$ replaced by $p^l$.*

PROOF. Apply the Hensel step inductively for $m = p, p^2, p^4, \ldots$. □

EXAMPLE 15.8 (continued). Let $p = 5$ and $f, g_1 = g^*, h_1 = h^*, s_1 = s^*, t_1 = t^*$ in $\mathbb{Z}[x]$ as in Example 15.8. Then $f \equiv g_1h_1 \bmod 25$ and $s_1g_1 + t_1h_1 \equiv 1 \bmod 25$, and another execution of Algorithm 15.10 yields

$$\begin{aligned} e_2 &\equiv 50x^2 - 50 \bmod 625, \\ q_2 &\equiv -225x + 300 \bmod 625, \quad r_2 \equiv -175 \bmod 625, \\ g_2 &\equiv x^3 + 182x^2 - x - 182 \bmod 625, \quad h_2 \equiv x - 182 \bmod 625, \\ b_2 &\equiv -225x^2 + 300x - 25 \bmod 625, \\ c_2 &\equiv 75x - 200 \bmod 625, \quad d_2 \equiv 275 \bmod 625, \\ s_2 &\equiv -267 \bmod 625, \quad t_2 \equiv 267x^2 - 312x - 176 \bmod 625. \end{aligned}$$

Then $s_2g_2 + t_2h_2 \equiv 1 \bmod 625$, which we don't actually need if we are only interested in a factorization modulo $p^4$, $f \equiv g_2h_2 \bmod 625$, and as in Example 9.24,

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

we see that 182 is the fourth root of 1 modulo 625 that is congruent to the starting solution 2 modulo 5. ◇

EXAMPLE 15.13. Let $p = 5$ and $f = x^4 - 1$, as in Example 15.8. Then $f$ has the roots $1, 2, -2, -1$ modulo 5, and hence

$$f \equiv (x-1)(x-2)(x+2)(x+1) \equiv (x^2+2x+2)(x^2-2x+2) \bmod 5.$$

We take $g_0 = x^2 + 2x + 2$ and $h_0 = x^2 - 2x + 2$ and compute $s_0 = -2x - 1$ and $t_0 = 2x - 1$ such that $s_0 g_0 + t_0 h_0 \equiv 1 \bmod 5$ by the Extended Euclidean Algorithm in $\mathbb{F}_5[x]$. Two Hensel steps then yield the following.

$$\begin{aligned}
&e_1 \equiv -5 \bmod 25,\\
&q_1 = 0, \quad r_1 \equiv 10x + 5 \bmod 25,\\
&g_1 \equiv x^2 - 8x + 7 \bmod 25, \quad h_1 \equiv x^2 + 8x + 7 \bmod 25,\\
&b_1 \equiv 5x^2 + 10 \bmod 25,\\
&c_1 \equiv -10x \bmod 25, \quad d_1 \equiv -10 \bmod 25,\\
&s_1 \equiv -2x + 9 \bmod 25, \quad t_1 \equiv 2x + 9 \bmod 25,\\
&e_2 \equiv 50x^2 - 50 \bmod 625,\\
&q_2 \equiv -100x \bmod 625, \quad r_2 \equiv 175x + 175 \bmod 625,\\
&g_2 \equiv x^2 - 183x + 182 \bmod 625, \quad h_2 \equiv x^2 + 183x + 182 \bmod 625,\\
&b_2 \equiv 125x^2 + 150 \bmod 625,\\
&c_2 \equiv -250x \bmod 625, \quad d_2 \equiv 200x + 100 \bmod 625,\\
&s_2 \equiv -202x - 91 \bmod 625, \quad t_2 \equiv 202x - 91 \bmod 625. \ \diamond
\end{aligned}$$

The following result corresponds to the uniqueness of Newton iteration (Theorem 9.27). It says essentially that two liftings modulo $p^l$ of the same factorization modulo $p$ coincide modulo $p^l$.

THEOREM 15.14 *Uniqueness of Hensel lifting.*
*Let $R$ be a ring (commutative, with 1), $p \in R$ not a zero divisor, $l \in \mathbb{N}_{\geq 1}$, and $g, h, g^*, h^*, s, t \in R[x]$ nonzero such that $sg + th \equiv 1 \bmod p$, the leading coefficients $\mathrm{lc}(g), \mathrm{lc}(h)$ are not zero divisors modulo $p$, the polynomials $g$ and $g^*$ have the same leading coefficient and the same degree, and coincide modulo $p$, and similarly for $h$ and $h^*$. If $gh \equiv g^* h^* \bmod p^l$, then $g \equiv g^* \bmod p^l$ and $h \equiv h^* \bmod p^l$.*

PROOF. We assume to the contrary that $g \not\equiv g^* \bmod p^l$ or $h \not\equiv h^* \bmod p^l$, and let $1 \leq i < l$ be maximal such that $p^i$ divides both $g^* - g$ and $h^* - h$. Thus $g^* - g = up^i$ and $h^* - h = vp^i$ for some $u, v \in R[x]$ such that $p \nmid u$ or $p \nmid v$. We may assume

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

that $p \nmid u$. Now

$$0 \equiv g^*h^* - gh = g^*(h^* - h) + h(g^* - g) = (g^*v + hu)p^i \bmod p^l.$$

Since $p$ is not a zero divisor, we have $p \mid p^{l-i} \mid (g^*v + hu)$. We denote by a bar the reduction modulo $p$. Then $\overline{sg} + \overline{t}\overline{h} = 1$, $\overline{g^*} = \overline{g}$, and $\overline{g^*}\overline{v} + \overline{h}\overline{u} = 0$. Thus

$$0 = \overline{t}(\overline{g^*}\overline{v} + \overline{h}\overline{u}) = \overline{t}\overline{g}\overline{v} + (1 - \overline{sg})\overline{u} = (\overline{t}\overline{v} - \overline{su})\overline{g} + \overline{u},$$

and hence $\overline{g} \mid \overline{u}$. Since $\mathrm{lc}(g) = \mathrm{lc}(g^*)$ and $\deg g = \deg g^*$, we have $\deg \overline{u} < \deg \overline{g}$. Since $\mathrm{lc}(\overline{g}) = \overline{\mathrm{lc}(g)}$ is not a zero divisor, neither is $\overline{g}$, and $\overline{u}$ is the zero polynomial. This contradicts our assumption that $p \nmid u$, and the claim is proved. □

The following consequence will be used in Section 16.5.

COROLLARY 15.15.
*Let $R$ be a Euclidean domain, $p \in R$ prime, $l \in \mathbb{N}_{>0}$, $f, g, u \in R[x]$ nonzero such that $p \nmid \mathrm{lc}(f)$, $f \bmod p$ is squarefree, $g$ divides $f$ in $R[x]$, and $u$ is monic, nonconstant, and divides $f$ modulo $p^l$ and $g$ modulo $p$. Then $u$ divides $g$ modulo $p^l$.*

PROOF. Let $h, v, w \in R[x]$ such that $f = gh \equiv uw \bmod p^l$ and $g \equiv uv \bmod p$. Since $f \bmod p$ is squarefree, so is $g \bmod p$, and $\gcd(u \bmod p, v \bmod p) = 1$ in $\mathbb{F}_p[x]$. Then Hensel's lemma 15.12 yields $u^*, v^* \in R[x]$ such that $u^* \equiv u \bmod p$, $v^* \equiv v \bmod p$, and $g \equiv u^*v^* \bmod p^l$. Now $uvh \equiv gh \equiv uw \bmod p$ implies $vh \equiv w \bmod p$. Thus $v^*h \equiv vh \equiv w \bmod p$ and $u^* \cdot (v^*h) \equiv gh = f \equiv uw \bmod p^l$. Together with the fact that $u$ and $v$ are coprime modulo $p$, Theorem 15.14 gives $u \equiv u^* \bmod p^l$, and finally $g \equiv uv^* \bmod p^l$. □

There is also an infinite version of Hensel's lemma. Let $p \in R$ be prime and $R_{(p)}$ the ***p*-adic completion** of $R$ (Section 9.6). If $R = \mathbb{Z}$, then this is the ring $\mathbb{Z}_{(p)}$ of ***p*-adic integers**, whose elements can be represented by "power series in $p$" of the form $\sum_{i \geq 0} a_i p^i$ with $0 \leq a_i < p$ for all $i \in \mathbb{N}$. If $R = F[y]$ for a field $F$ and $p = y$, then $R_{(p)} = F[[y]]$ is the ring of formal power series in $y$ with coefficients in $F$.

THEOREM 15.16 *Hensel's lemma, infinite version.*
*Assuming the input specification of Algorithm 15.10 for a prime $m = p \in R$, there exist polynomials as in the output specification, but with all congruences modulo $m^2$ replaced by equalities in $R_{(p)}$.*

The general multivariate Newton iteration works as follows. One has $n$ functions $\varphi = (\varphi_1, \ldots, \varphi_n)$ in $n$ variables $y_1, \ldots, y_n$, and from an approximation $a \in R^n$ to a

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

common root of $\varphi_1,\dots,\varphi_n$ obtains a better one $a^*$ as $a^* = a - J^{-1}(a)f(a)$, where $J = (\partial\varphi_i/\partial y_j)_{1\le i,j\le n} \in R[y_1,\dots,y_n]^{n\times n}$ is the **Jacobian** of $\varphi$.

We have seen in Example 15.8 that Hensel lifting generalizes Newton iteration. But Hensel lifting can also be considered as a special case of multivariate Newton iteration, as follows. We regard the coefficients of $g$ and $h$ in a factorization $f = gh$ as indeterminates. Comparing the coefficients of $x^{n-1},\dots,x,1$ on both sides, we obtain $n$ equations for the $n$ unknown coefficients, and a common solution to those equations corresponds to a factorization of $f$. Invertibility of $J$ is equivalent to coprimality of $g$ and $h$ (Exercise 15.21).

## 15.5. Multifactor Hensel lifting

In this section, we will employ the Hensel step algorithm 15.10 to lift a factorization into arbitrarily many factors. Let $R$ be a ring, $p \in R$, and $g,h \in R[x]$. We say that $g$ and $h$ are **Bézout-coprime** modulo $p$ if there exist $s,t \in R[x]$ such that $sg + th \equiv 1 \bmod p$. If $R/\langle p\rangle$ is a field, then this just means that $g \bmod p$ and $h \bmod p$ are coprime in the usual sense.

ALGORITHM 15.17 Multifactor Hensel lifting.

Input: An element $p$ in a ring $R$ (commutative, with 1), $f \in R[x]$ of degree $n \ge 1$ such that $\mathrm{lc}(f)$ is a unit modulo $p$, monic nonconstant polynomials $f_1,\dots,f_r \in R[x]$ that are pairwise Bézout-coprime modulo $p$ and satisfy $f \equiv \mathrm{lc}(f)f_1\cdots f_r \bmod p$, and $l \in \mathbb{N}$.

Output: Monic polynomials $f_1^*,\dots,f_r^* \in R[x]$ with $f \equiv \mathrm{lc}(f)f_1^*\cdots f_r^* \bmod p^l$ and $f_i^* \equiv f_i \bmod p$ for all $i$.

1. **if** $r = 1$ **then** compute $f_1^* \in R[x]$ with $f \equiv \mathrm{lc}(f)f_1^* \bmod p^l$ and **return** $f_1^*$
2. $k \longleftarrow \lfloor r/2\rfloor, \quad d \longleftarrow \lceil \log_2 l\rceil$
3. compute $g_0,h_0 \in R[x]$ such that $g_0 \equiv \mathrm{lc}(f)f_1\cdots f_k \bmod p$ and $h_0 \equiv f_{k+1}\cdots f_r \bmod p$
4. compute $s_0,t_0 \in R[x]$ such that $s_0g_0 + t_0h_0 \equiv 1 \bmod p$, $\deg s_0 < \deg h_0$, and $\deg t_0 < \deg g_0$, using the Extended Euclidean Algorithm if $R/\langle p\rangle$ is a field and Exercise 15.29 otherwise
5. **for** $j = 1,\dots,d$ **do**
6. **call** the Hensel step algorithm 15.10 with $m = p^{2^{j-1}}$ to lift the congruences $f \equiv g_{j-1}h_{j-1}$ and $s_{j-1}g_{j-1} + t_{j-1}h_{j-1} \equiv 1$ modulo $p^{2^{j-1}}$ to congruences $f \equiv g_jh_j$ and $s_jg_j + t_jh_j \equiv 1$ modulo $p^{2^j}$
7. $g \longleftarrow g_d, \quad h \longleftarrow h_d$

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

8. **call** the algorithm recursively to compute $f_1^*, \ldots, f_k^* \in R[x]$ satisfying $g \equiv \mathrm{lc}(g) f_1^* \cdots f_k^* \bmod p^l$ and $f_i^* \equiv f_i \bmod p$ for $1 \le i \le k$
9. **call** the algorithm recursively to compute $f_{k+1}^*, \ldots, f_r^* \in R[x]$ satisfying $h \equiv f_{k+1}^* \cdots f_r^* \bmod p^l$ and $f_i^* \equiv f_i \bmod p$ for $k < i \le r$
10. **return** $f_1^*, \ldots, f_r^*$

THEOREM 15.18.
*Algorithm 15.17 works correctly as specified.*

*(i) If $R = \mathbb{Z}$, $p \in \mathbb{N}$ is prime, $\|f\|_\infty < p^l$, and $\|f_i\|_\infty < p$ for all $i$, then the algorithm takes*

$$O\Big((\mathsf{M}(n)\mathsf{M}(l\mu) + \mathsf{M}(n)\log n \cdot \mathsf{M}(\mu) + n\mathsf{M}(\mu)\log\mu)\log r\Big)$$

*or $O^\sim(nl\mu)$ word operations, where $\mu = \log p$.*

*(ii) If $R = F[y]$ for a field $F$, $p$ is irreducible, $\deg_y f < l \deg_y p$, and $\deg_y f_i < \deg_y p$ for all $i$, then the algorithm takes*

$$O\Big((\mathsf{M}(n)\mathsf{M}(l\mu) + \mathsf{M}(n)\log n \cdot \mathsf{M}(\mu) + n\mathsf{M}(\mu)\log\mu)\log r\Big)$$

*or $O^\sim(nl\mu)$ arithmetic operations in $F$, where $\mu = \deg_y p$.*

PROOF. Correctness follows from Theorem 15.11 by induction on $j$ and on $r$. We only prove the running time estimate (i); case (ii) is similar and left as Exercise 15.22. Let $n_i = \deg f_i$ for $1 \le i \le r$, and let $T(n_1, \ldots, n_r)$ denote the running time of the algorithm in word operations. For $r = 1$, we first compute in step 1 $\mathrm{lc}(f)^{-1} \bmod p$ by the Extended Euclidean Algorithm, taking $O(\mathsf{M}(\mu)\log\mu)$ word operations, by Corollary 11.11. Then we use the Newton iteration algorithm 9.10 to lift the inverse modulo $p^l$, in time $O(\mathsf{M}(l\mu))$, by Theorem 9.12. Multiplying all coefficients of $f$ by $\mathrm{lc}(f)^{-1}$ modulo $p^l$ takes $O(n\mathsf{M}(l\mu))$ word operations. We conclude that $T(n) \in O(n\mathsf{M}(l\mu) + \mathsf{M}(\mu)\log\mu)$.

Now let $r \ge 2$. By Lemma 10.4, the cost for step 3 is $O(\mathsf{M}(n)\log r)$ additions and multiplications in the field $\mathbb{F}_p$. Step 4 takes $O(\mathsf{M}(n)\log n)$ additions and multiplications in $\mathbb{F}_p$ plus $O(n)$ divisions, by Corollary 11.9. By Corollary 11.11, the cost for steps 3 and 4 in word operations is therefore $O(\mathsf{M}(n)\log n \cdot \mathsf{M}(\mu) + n\mathsf{M}(\mu)\log\mu)$.

By Exercise 9.7, we can reduce all coefficients of $f$ modulo $p, p^2, p^4, \ldots, p^{2^{d-1}}$ with $O(n\mathsf{M}(l\mu))$ word operations. Theorem 15.11 implies that the cost for the $j$th execution of step 6 is $O(\mathsf{M}(n)\mathsf{M}(2^j\mu))$ word operations. Summing this over $j$, the superlinearity of $\mathsf{M}$ (Section 8.3) and the geometric sum $\sum_{1 \le j \le d} 2^j \le 4l$ imply that the overall cost for steps 5 and 6 is $O(\mathsf{M}(n)\mathsf{M}(l\mu))$ word operations.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

We may assume that $r$ is a power of two. Steps 8 and 9 take $T(n_1,\dots,n_k)$ and $T(n_{k+1},\dots,n_r)$ word operations, respectively. Adding costs leads to the recursive inequalities

$$\begin{aligned} T(n) &\le c(n\mathsf{M}(l\mu)+\mathsf{M}(\mu)\log\mu),\\ T(n_1,\dots,n_r) &\le T(n_1,\cdots,n_k)+T(n_{k+1},\dots,n_r)\\ &\quad +c\Big(\mathsf{M}(n)\mathsf{M}(l\mu)+\mathsf{M}(n)\log n\cdot\mathsf{M}(\mu)+n\mathsf{M}(\mu)\log\mu\Big), \end{aligned}$$

for some positive constant $c$. Now

$$T(n_1,\dots,n_r)\le c\Big(\mathsf{M}(n)\mathsf{M}(l\mu)+\mathsf{M}(n)\log n\cdot\mathsf{M}(\mu)+n\mathsf{M}(\mu)\log\mu\Big)(1+\log r),$$

where $n=n_1+\cdots+n_r$, follows from the superlinearity of $\mathsf{M}$ (Section 8.3) by induction on $r$. □

If we balance degrees in step 2, as discussed in Section 10.1, then the factor $\log r$ in the timing estimate above may be replaced by the entropy $H(n_1/n,\dots,n_r/n)$, where $n_i=\deg f_i$ for $1\le i\le r$.

EXAMPLE 15.13 (continued). We let $l=4$. The factorization of $f=x^4-1$ into monic irreducible polynomials modulo 5 is

$$x^4-1\equiv f_1f_2f_3f_4=(x-1)(x-2)(x+2)(x+1) \bmod 5.$$

Algorithm 15.17 computes $k=2$ and $d=2$ in step 2 and

$$g\equiv f_1f_2\equiv x^2+2x+2 \bmod 5,\quad h\equiv f_3f_4\equiv x^2-2x+2 \bmod 5$$

in step 3, and the Extended Euclidean Algorithm in $\mathbb{F}_5[x]$ yields $s=-2x-1$ and $t=2x-1$ in step 4. In Example 15.13, we have already performed the computations of steps 5 and 6, and we find $g=x^2-183x+182$ and $h=x^2+183x+182$ in step 7. In steps 8 and 9, we recursively lift the factorizations

$$g\equiv f_1f_2=(x-1)(x-2) \bmod 5,\quad h\equiv f_3f_4=(x+2)(x+1) \bmod 5$$

to factorizations

$$g\equiv f_1^*f_2^*=(x-1)(x-182) \bmod 625,\quad h\equiv f_3^*f_4^*=(x+182)(x+1) \bmod 625.$$

Thus $f\equiv(x-1)(x-182)(x+182)(x+1) \bmod 625$, and the fourth roots of unity modulo $625=5^4$ are $\pm1$ and $\pm182$. ◇

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

## 15.6. Factoring using Hensel lifting: Zassenhaus' algorithm

Let $R = \mathbb{Z}$ and $f \in \mathbb{Z}[x]$ be a squarefree primitive polynomial of degree $n$. We will now replace step 3 of the big prime algorithm 15.2 by factoring modulo a "small" prime $p \in \mathbb{N}$ and subsequent Hensel lifting. As in Algorithm 15.2, we will choose $p$ in such a way that $f \bmod p$ is squarefree and of the same degree as $f$, so that $p$ does not divide $\operatorname{res}(f, f')$. The bound on the resultant from Theorem 6.23 implies that this is an easy task; the details are given later in Corollary 18.12.

ALGORITHM 15.19 Factorization in $\mathbb{Z}[x]$ (prime power version).

Input: A squarefree primitive polynomial $f \in \mathbb{Z}[x]$ of degree $n \geq 1$ with $\operatorname{lc}(f) > 0$ and max-norm $\|f\|_\infty = A$.

Output: The irreducible factors $\{f_1, \ldots, f_k\} \subseteq \mathbb{Z}[x]$ of $f$.

1. **if** $n = 1$ **then return** $\{f\}$
   $b \longleftarrow \operatorname{lc}(f)$, $\quad B \longleftarrow (n+1)^{1/2} 2^n A b$,
   $C \longleftarrow (n+1)^{2n} A^{2n-1}$, $\quad \gamma \longleftarrow \lceil 2 \log_2 C \rceil$

2. **repeat** choose a prime $p \leq 2\gamma \ln \gamma$, $\quad \overline{f} \longleftarrow f \bmod p$
   **until** $p \nmid b$ and $\gcd(\overline{f}, \overline{f}') = 1$ in $\mathbb{F}_p[x]$
   $l \longleftarrow \lceil \log_p (2B+1) \rceil$

3. { modular factorization }
   compute $h_1, \ldots, h_r \in \mathbb{Z}[x]$ of max-norm at most $p/2$ that are nonconstant, monic, and irreducible modulo $p$, such that $f \equiv b h_1 \cdots h_r \bmod p$

4. { Hensel lifting }
   **call** Algorithm 15.17 to compute a factorization $f \equiv b g_1 \cdots g_r \bmod p^l$ with monic polynomials $g_1, \ldots, g_r \in \mathbb{Z}[x]$ of max-norm at most $p^l/2$ such that $g_i \equiv h_i \bmod p$ for $1 \leq i \leq r$

5. { initialize the index set $T$ of modular factors still to be treated, the set $G$ of factors found, and the polynomial $f^*$ still to be factored }
   $T \longleftarrow \{1, \ldots, r\}$, $\quad s \longleftarrow 1$, $\quad G \longleftarrow \emptyset$, $\quad f^* \longleftarrow f$

6. { factor combination }
   **while** $2s \leq \#T$ **do**

7. **for** all subsets $S \subseteq T$ of cardinality $\#S = s$ **do**

8. compute $g^*, h^* \in \mathbb{Z}[x]$ with max-norm at most $p^l/2$ satisfying $g^* \equiv b \prod_{i \in S} g_i \bmod p^l$ and $h^* \equiv b \prod_{i \in T \setminus S} g_i \bmod p^l$

9. **if** $\|g^*\|_1 \|h^*\|_1 \leq B$ **then**
   $T \longleftarrow T \setminus S$, $\quad G \longleftarrow G \cup \{\operatorname{pp}(g^*)\}$,
   $f^* \longleftarrow \operatorname{pp}(h^*)$, $\quad b \longleftarrow \operatorname{lc}(f^*)$
   break the loop 7 and **goto** 6

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

10. $s \longleftarrow s+1$

11. **return** $G \cup \{f^*\}$

There are several ways to find suitable primes in step 2: we might try the small primes $2,3,5,\ldots$ one after the other, or we might use a single precision prime just below the processor's word length from a precalculated list. Both approaches work well in practice, but do not yield a generally valid result since for some particular input all primes from any fixed list might divide the discriminant. Another alternative which provably works is to choose $p$ randomly; the required number theoretic arguments will be discussed in Section 18.4.

THEOREM 15.20.
*Algorithm 15.19 works correctly. We have $\gamma \in O(n\log(nA))$, and the expected cost of steps 2 and 3 is*

$$O\Big(\gamma\log^2\gamma\,\mathrm{loglog}\,\gamma + (\mathsf{M}(n^2)+\mathsf{M}(n)\log\gamma)\log n\cdot\mathsf{M}(\log\gamma)\,\mathrm{loglog}\,\gamma\Big)$$

*or $O^\sim(n^2+n\log A)$ word operations. Step 4 takes $O(\mathsf{M}(n)\mathsf{M}(n+\log A)\log n)$ or $O^\sim(n^2+n\log A)$ word operations. The cost for one execution of steps 8 and 9 is*

$$O((\mathsf{M}(n)\log n + n\log(n+\log A))\mathsf{M}(n+\log A)) \text{ or } O^\sim(n^2+n\log A)$$

*word operations, and there are at most $2^{n+1}$ iterations.*

PROOF. The correctness proof of Theorem 15.3 carries over with the following modifications. We replace the congruence in (2) by $f^* \equiv b\prod_{i\in T} g_i \bmod p^l$. In one part of that proof we assume that the condition in step 9 is false for all subsets $S \subseteq T$ of cardinality $s$, but that $f^*$ has an irreducible factor $g \in \mathbb{Z}[x]$ with $\mu(g) = s$, and the fact that $\mathbb{F}_p[x]$ is a UFD yields a set $S \subseteq T$ of cardinality $s$ such that the condition in step 9 is true for that particular subset. Now $\mathbb{Z}_{p^l}[x]$ is not a UFD in general (it even has nonzero zero divisors), and we have to replace the argument by unique factorization in $\mathbb{F}_p[x]$ plus an appeal to the uniqueness of Hensel lifting (Theorem 15.14). Namely, let $h = f^*/g$ and $S \subseteq T$ with $\#S = s$ be such that $\mathrm{lc}(h)g \equiv b\prod_{i\in S} h_i \bmod p$ and $\mathrm{lc}(g)h \equiv b\prod_{i\in T\setminus S} h_i \bmod p$. Now for that same subset $S$, let $g^* \equiv b\prod_{i\in S} g_i \bmod p^l$ and $h^* \equiv b\prod_{i\in T\setminus S} g_i \bmod p^l$. Thus $bf^* \equiv \mathrm{lc}(h)g\cdot\mathrm{lc}(g)h \bmod p^l$ and $bf^* \equiv g^*h^* \bmod p^l$ are both liftings of the same factorization of $bf^*$ modulo $p$, and the uniqueness of Hensel lifting (Theorem 15.14) implies that $\mathrm{lc}(h)g \equiv g^* \bmod p^l$ and $\mathrm{lc}(g)h \equiv h^* \bmod p^l$. Now $B < p^l/2$, by the choice of $l$, and as in the proof of Theorem 15.3, we arrive at the contradiction that $\|g^*\|_1\|h^*\|_1 \le B$ holds in step 9.

Corollary 18.12 says that with $O(\gamma\log^2\gamma\,\mathrm{loglog}\,\gamma)$ word operations, we can find a random $p$ in step 2, and that $p$ divides $\mathrm{disc}(f)$ with probability at most $1/2$.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

Now $b \mid \operatorname{disc}(f)$, by Exercise 6.41, and Lemma 15.1 implies that the expected number of iterations of step 2 is at most two. The length of $p$ is in $O(\log\gamma)$, the cost for reducing all coefficients of $f$ modulo $p$ is $O(n\log A \cdot \log\gamma)$ word operations, and the gcd takes $O(\mathsf{M}(n)\log n \cdot \mathsf{M}(\log\gamma)\operatorname{loglog}\gamma)$ word operations. The cost estimate for step 4 follows from Theorem 15.18 with $\mu = \log p \in O(\log\gamma)$, using $\log n \log\gamma \in O(n + \log A)$ and $\log\gamma \operatorname{loglog}\gamma \in O(n + \log A)$, and the rest of the analysis is as in the proof of Theorem 15.3. □

EXAMPLE 15.4 (continued). Let $f = 6x^4 + 5x^3 + 15x^2 + 5x + 4 \in \mathbb{Z}[x]$ be as in Example 15.4, and suppose that we choose $p = 5$ in step 2, so that $p$ divides neither the leading coefficient nor the discriminant of $f$. Then $l = \lceil \log_5(2B+1) \rceil = 6$. In step 3, we obtain the modular factorization

$$f \equiv b h_1 h_2 h_3 h_4 = 1 \cdot (x-1)(x+1)(x-2)(x+2) \bmod 5.$$

In step 4, Algorithm 15.17 lifts it to the modular factorization

$$f \equiv b g_1 g_2 g_3 g_4 = 6 \cdot (x-5136)(x-984)(x-72)(x-6828) \bmod 5^6.$$

Among the subsets $S \subseteq \{1,2,3,4\}$ of cardinality $s = 1$, none leads to a factorization. For $S = \{1,3\}$ we compute

$$\begin{aligned} g^* &\equiv b g_1 g_3 = 6(x-5136)(x-72) \equiv 6x^2 + 2x + 2 \bmod 5^6, \\ h^* &\equiv b g_2 g_4 = 6(x-984)(x-6828) \equiv 6x^2 + 3x + 12 \bmod 5^6, \end{aligned}$$

in step 8 and obtain $\|g^*\|_1 \|h^*\|_1 \leq B$ and $g^* h^* = b f^*$ in step 9. Thus $\operatorname{pp}(g^*) = 3x^2 + x + 1$ and $\operatorname{pp}(h^*) = 2x^2 + x + 4$ are the irreducible factors of $f$ in $\mathbb{Z}[x]$, as in Example 15.4. ◇

Steps 2 and 3 of the big prime algorithm 15.2 take about $O^\sim(n^3 + \log^3 A)$ word operations, while the cost for the corresponding steps 2 through 4 in the prime power algorithm 15.19 is only about $O^\sim(n^2 + n\log A)$ word operations. If $n \approx \log A$, then the former is roughly cubic in $n$, while the latter is only quadratic. Like Algorithm 15.2, Zassenhaus' algorithm has exponential running time in the worst case. Nevertheless, the algorithm works well in practice and should be used in the complete factorization algorithm 15.5; this is confirmed by experiments in Section 15.7. Collins (1979) showed, under a plausible but unproven hypothesis, that the algorithm uses polynomial time "on the average".

There is one further advantage of Zassenhaus' algorithm over the big prime approach: since the Mignotte bound determining $l$ is usually far too large, we may interleave Hensel lifting and factor combination, as follows. We first lift the factorization modulo $p^{l^*}$ for some $l^* < l$, then check whether some of the modular factors are true factors in $\mathbb{Z}[x]$, remove these from $f$, and then lift the remaining

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

factorization modulo some higher power of $p$. This is iterated until all factors of $f$ are found, which—if we are lucky—may happen before $p^l$ is reached. A natural choice for these $l^*$ are consecutive powers of 2, starting with the smallest such $l^*$ which is at least $\|f\|_\infty$.

If we are only interested in computing all integral or rational roots of a given polynomial $f \in \mathbb{Z}[x]$, then we can greatly simplify Zassenhaus' algorithm 15.19. We have already discussed an approach via factoring modulo a big prime in Section 14.5. For the prime power approach, we modify Algorithm 15.19 in the following respects. Firstly, since we are only interested in linear factors, we may replace the bound $2B$ by $2nb(A^2+A)$ if the latter is smaller, as in the big prime algorithm 14.17. Secondly, we need only compute the linear factors of $f$ modulo $p$ in step 3 (executing the distinct-degree and equal-degree factorization only for degree 1), and finally—and most importantly—the whole factor combination stage may be replaced by a simple check whether the linear factors modulo $p^l$ are linear factors in $\mathbb{Z}[x]$.

THEOREM 15.21.
*Given a nonconstant squarefree primitive polynomial $f \in \mathbb{Z}[x]$, we can compute all its rational roots with an expected number of*

$$O\Big(n\log(nA)(\log\log A)^2\log\log\log A + \mathsf{M}(n)\log n\cdot\mathsf{M}(\log(n\log A))\log(n\log A)\log\log(n\log A) + n^2\mathsf{M}(\log(nA))\Big)$$

*or $O^\sim(n^2\log A)$ word operations.*

PROOF. Let $\gamma = \log C \in O(n\log(nA))$. The expected cost for step 2 of Algorithm 15.19, modified as described above, is

$$O(\gamma\log^2\gamma\log\log\gamma + n\log A\cdot\log\gamma + \mathsf{M}(n)\log n\cdot\mathsf{M}(\log\gamma)\log\log\gamma)$$

word operations, as in the proof of Theorem 15.20. The expected cost for computing all (monic) linear factors of $f$ mod $p$ in step 3 is $O(\mathsf{M}(n)\log n\log(n\gamma))$ arithmetic operations in $\mathbb{F}_p$, by Corollary 14.16, or

$$O(\mathsf{M}(n)\log n\log(n\gamma)\mathsf{M}(\log\gamma)\log\log\gamma)$$

word operations. We take all monic linear factors plus the remaining monic cofactor of $f$ mod $p$ as inputs to the Hensel lifting in step 4. The cost for step 4 is

$$O\Big((\mathsf{M}(n)\mathsf{M}(\log(nA)) + \mathsf{M}(n)\log n\cdot\mathsf{M}(\log\gamma) + n\mathsf{M}(\log\gamma)\log\log\gamma)\log n\Big)$$

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

word operations, by Theorem 15.18 with $\mu = \log p \in O(\log \gamma)$. For each linear factor $bx - c \in \mathbb{Z}[x]$ dividing $f$ modulo $p^l$, with $b = \mathrm{lc}(f)$, we compute the corresponding cofactor $v \in \mathbb{Z}[x]$ such that $(bx-c)v \equiv bf \bmod p^l$ in step 8, at a cost of $O(n\mathsf{M}(\log(nA)))$ word operations, as in the proof of Theorem 14.18. There are at most $n$ modular factors, and hence the total cost for checking all of them is $O(n^2\mathsf{M}(\log(nA)))$ word operations. We obtain an overall cost of

$$O(\gamma \log^2 \gamma \log\log \gamma + \mathsf{M}(n) \log n \log(n\gamma)\, \mathsf{M}(\log \gamma) \log\log \gamma + n^2 \mathsf{M}(\log(nA)))$$

word operations, and the claim follows. □

Due to the checking in the last step, the overall asymptotic running time of the prime power algorithm for root finding is about the same as for the big prime variant (Algorithm 14.17). However, the modular factorization stage of the former takes about $O^{\sim}(n \log^2 A)$ word operations, while the cost for the modular factorization stage, including Hensel lifting, is only about $O^{\sim}(n \log A)$ for the prime power algorithm. In practice, one would expect that most of the "false" roots exceed the trailing coefficient in absolute value, and would test the remaining trial roots modulo some other small primes first; this should rule out most of those which are not roots in $\mathbb{Q}$. The hope is that then there remain only few false roots.

With minor changes, Zassenhaus' algorithm 15.19 can be adapted to factor bivariate polynomials over a field $F$ with **effective univariate factorization**, so that we know how to factor univariate polynomials over $F$ (for example, $F = \mathbb{Q}$ or $F = \mathbb{F}_q$ for a prime power $q$). The degree in $y$ of a polynomial $f \in F[x,y]$ plays the role of the max-norm, and a bound for possible factors of $f$ is much simpler than in the integer case: divisors of $f$ never have larger degree than $f$ does. "Primitive" now is with respect to the variable $x$, so that $\mathrm{cont}_x(f) = 1$. Moreover, we require $f$ to have a trivial gcd with its derivative with respect to $x$. This implies that $f$ is squarefree. The converse is true in characteristic zero; see Exercise 15.25 for a counterexample in positive characteristic.

ALGORITHM 15.22 Bivariate factorization (prime power version).
Input: A primitive polynomial $f \in R[x] = F[x,y]$, where $R = F[y]$ for a field $F$ with at least $4nd$ elements and effective univariate factorization, such that $\deg_x f = n \geq 1$, $\deg_y f = d$, and $\gcd(f, \partial f/\partial x) = 1$ in $F(y)[x]$.
Output: The irreducible factors $\{f_1, \ldots, f_k\} \subseteq F[x,y]$ of $f$.

1. **if** $n = 1$ **then return** $f$
   $b \longleftarrow \mathrm{lc}_x(f)$
   choose $U \subseteq F$ of cardinality $\#U \geq 4nd$
2. **repeat** randomly choose an element $u \in U$, $\quad \overline{f} \longleftarrow f(x,u)$
   **until** $b(u) \neq 0$ and $\gcd(\overline{f}, \overline{f}') = 1$ in $F[x]$
   $l \longleftarrow d + 1 + \deg b$

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

3. { modular factorization }
use the univariate factorization algorithm to compute a factorization $f \equiv bh_1 \cdots h_r \bmod (y-u)$ in $(F[y]/\langle y-u\rangle)[x] \cong F[x]$ with distinct monic irreducible $h_1, \ldots, h_r \in F[x]$

4. { Hensel lifting }
**call** Algorithm 15.17 to compute a factorization $f \equiv bg_1 \cdots g_r \bmod (y-u)^l$ with polynomials $g_1, \ldots, g_r \in F[x,y]$ that are monic with respect to $x$ such that $\deg_y g_i < l$ and $g_i(x,u) = h_i$ for $1 \le i \le r$

5. { initialize the index set $T$ of modular factors still to be treated, the set $G$ of factors found, and the polynomial $f^*$ still to be factored }
$T \longleftarrow \{1, \ldots, r\}, \quad s \longleftarrow 1, \quad G \longleftarrow \emptyset, \quad f^* \longleftarrow f$

6. { factor combination }
**while** $2s \le \#T$ **do**

7. **for** all subsets $S \subseteq T$ of cardinality $\#S = s$ **do**

8. compute $g^*, h^* \in F[x,y]$ of degree less than $l$ in $y$ satisfying $g^* \equiv b\prod_{i\in S} g_i \bmod (y-u)^l$ and $h^* \equiv b\prod_{i\in T\setminus S} g_i \bmod (y-u)^l$

9. **if** $\deg(g^*h^*) = \deg(bf^*)$ **then**
$T \longleftarrow T \setminus S, \quad G \longleftarrow G \cup \{\mathrm{pp}_x(g^*)\},$
$f^* \longleftarrow \mathrm{pp}_x(h^*), \quad b \longleftarrow \mathrm{lc}_x(f^*)$
break the loop 7 and **goto** 6

10. $s \longleftarrow s+1$

11. **return** $G \cup \{f^*\}$

The following analysis of the algorithm is in Exercise 15.24.

THEOREM 15.23.
*Algorithm 15.22 works correctly. The expected cost of step 2 is $O(nd + \mathsf{M}(n)\log n)$ or $O^\sim(nd)$ arithmetic operations in $F$, and step 4 takes $O(\mathsf{M}(n)\log n(\log n + \mathsf{M}(d))$ or $O^\sim(nd)$ operations. The number of field operations for one iteration of steps 8 and 9 is $O((n\log d + \mathsf{M}(n)\log n)\mathsf{M}(d))$ or $O^\sim(nd)$, and there are at most $2^{n+1}$ iterations. If $F = \mathbb{F}_q$ is a finite field with $q$ elements, then the expected number of operations in $\mathbb{F}_q$ for step 3 is $O(\mathsf{M}(n^2)\log n + \mathsf{M}(n)\log n \log q)$ or $O^\sim(n^2 + n\log q)$.*

When $F = \mathbb{F}_q$ is a "small" finite field, then we have a problem in step 1. There are several ways to circumvent this. We may employ a big prime algorithm,

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

with a worse cost estimate for the modular factorization stage. Or we may factor $f$ modulo a nonlinear irreducible $m \in \mathbb{F}_q[y]$ of degree $O(\log(nd))$ in step 3 instead of modulo $y-u$, and lift this to a factorization modulo a sufficiently high power of $m$ in step 4. This increases the timings of steps 2 and 3 by a factor of at most $O^{\sim}(\log^2(nd))$. Or we perform a field extension of degree $O(\log(nd))$ and employ the algorithm over the larger field, thereby multiplying a factor of $O^{\sim}(\log^2(nd))$ to all timings. However, irreducible factors of $f$ in $\mathbb{F}_q[x,y]$ may split over the larger field. In some applications, such a finer factorization may be advantageous, but if factors over $\mathbb{F}_q$ are needed, one has to take care of this separately if necessary. For example, if $g \in \mathbb{F}_{q'}[x,y]$ is an irreducible factor of $f$ over the larger field $\mathbb{F}_{q'}$, then $g^{(q'-1)/(q-1)} \in \mathbb{F}_q[x,y]$ (the norm of $g$) is a power of an irreducible factor of $f$ over $\mathbb{F}_q$.

To factor an arbitrary polynomial $f \in F[x,y]$, we proceed similarly as described after Algorithm 15.2, with one notable difference: If $F = \mathbb{F}_q$ is a finite field, then $\mathbb{F}_q(y)$ is not a perfect field, and we cannot literally use the algorithms of Section 14.6 for squarefree factorization. For example, if $p = \operatorname{char}\mathbb{F}_q$, then the polynomial $f = x^p - y$ is irreducible, but has derivative $\partial f/\partial x = 0$. If we exchange the roles of $x$ and $y$, then $\partial f/\partial y = -1$ and $\gcd(f, \partial f/\partial y) = 1$ in $\mathbb{F}_q(x)[y]$, and the algorithm can be applied. If both partial derivatives $\partial f/\partial x$ and $\partial f/\partial y$ vanish, as for $f = x^p - y^p$, then $f$ is a $p$th power, as in the univariate case (here $f = (x-y)^p$), and it is sufficient to factor $f^{1/p}$. Here is the analog of Algorithm 15.5; see Exercise 15.25 for a correctness proof.

ALGORITHM 15.24 Bivariate polynomial factorization.

Input: A nonconstant polynomial $f \in F[x,y]$ with $\deg_x f = n$ and $\deg_y f = d$, where $F$ is a field with effective univariate factorization.

Output: A constant $a \in F$ and a set of pairs $\{(f_1,e_1),\ldots,(f_k,e_k)\}$ such that the $f_i \in F[x,y]$ are irreducible and distinct, the $e_i \in \mathbb{N}$ are positive, and $f = a f_1^{e_1} \cdots f_r^{e_r}$.

1. $G \longleftarrow \emptyset, \quad c_x \longleftarrow \operatorname{cont}_x(f), \quad c_y \longleftarrow \operatorname{cont}_y(f), \quad g \longleftarrow \dfrac{f}{c_x c_y}$

2. compute the irreducible factorizations of $c_x$ and $c_y$ in $F[y]$ and $F[x]$, respectively, and add them to $G$

3. $g_x \longleftarrow \partial g/\partial x, \quad u \longleftarrow \dfrac{g}{\gcd(g,g_x)}$
   $g_y \longleftarrow \partial g/\partial y, \quad v \longleftarrow \dfrac{g}{\gcd(g,g_y)}, \quad w \longleftarrow \dfrac{u}{\gcd(u,v)}$
   { gcd denotes the greatest common divisor in $F[x,y]$ }

4. **call** Algorithm 15.22 with the roles of $x$ and $y$ exchanged to compute the set $V \subseteq F[x,y]$ of all irreducible factors of $v$
   **call** Algorithm 15.22 to compute the set $W \subseteq F[x,y]$ of all irreducible factors of $w$

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

5. **for** all $v \in V \cup W$ **do**
   determine the multiplicity $e$ of $v$ in $g$ by trial division
   $G \longleftarrow G \cup \{(v,e)\}, \quad g \longleftarrow \frac{g}{v^e}$

6. **if** $F$ is finite of characteristic $p$ and $g \notin F$ **then**
   **call** the algorithm recursively with input $g^{1/p}$, yielding $a \in F$ and a set of pairs $H$
   $a \longleftarrow a^p$
   **for** each pair $(g,e) \in H$ **do** $G \longleftarrow G \cup \{(g,ep)\}$
   **else** $a \longleftarrow g$

7. **return** $a$ and $G$

We conclude this section with a recapitulation of the algorithms which employ one of our three modular computation schemes: big prime, small primes, or prime power modular algorithms. They are given in Table 15.5.

| computation problem | big prime | small primes | prime power |
|---|---|---|---|
| determinant | Section 5.5 | Algorithm 5.10, Exercise 5.32 | |
| linear system solving | Exercise 5.33 | | |
| polynomial gcd | Algorithms 6.28, 6.34 | Algorithms 6.36, 6.38 | EZ-GCD (Notes 15.6) |
| polynomial EEA | | Algorithms 6.57, 6.59 | |
| integer multiplication | Exercise 8.36 | Algorithm 8.25 | |
| polynomial multiplication | | Algorithms 8.16, 8.20, Exercise 5.34 | |
| polynomial division | Exercise 9.14 | Exercise 10.21 | Algorithm 9.3 |
| roots of integers | | | Section 9.5 |
| root finding | Algorithm 14.17 | | Theorem 15.21 |
| squarefree decomposition | | Exercises 15.26, 15.31 | Exercise 15.27 |
| polynomial factorization | Algorithm 15.2 | | Algorithms 15.19, 15.22, 16.22 |

TABLE 15.5: Modular algorithms.

For all problems listed, there exist big prime algorithms, and in most cases also small primes and prime power algorithms, although we did not discuss all of them. Usually the small primes or prime power variants are more efficient, both in theory and in practice, than either the big prime approach or a direct computation. To apply the prime power approach, the computational problem needs to be described

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

in terms of "equations" which are then lifted. Some problems are badly suited for the small primes approach, such as polynomial factorization.

## 15.7. Implementations

In this section, we continue the description of NTL and BIPOLAR from Section 9.7, the focus now being on polynomial factorization. All experiments with NTL were done in version 1.5.

BIPOLAR is a C++ library for polynomial factorization in $\mathbb{F}_2[x]$. It contains algorithms for squarefree decomposition (Section 14.6), distinct-degree factorization (Section 14.2), and equal-degree factorization (Section 14.3). Experiments show that for random polynomials, the distinct-degree factorization stage is by far the dominant part of the whole computation.

Both in theory and in practice, a gcd computation is more costly than a multiplication or a division with remainder. Since a polynomial of degree $n$ does not have irreducible factors of all degrees between 1 and $n$ (on average, the number of irreducible factors is $O(\log n)$; see Notes 15.7), most gcds in the distinct-degree factorization algorithm 14.3 are equal to 1. BIPOLAR uses a blocking strategy to reduce the number of gcds, at the expense of additional—but cheaper—modular multiplications. The range $\{1, \ldots, n\}$ of possible degrees for the irreducible factors of $f$ is partitioned into disjoint intervals, and then for each interval $I$—proceeding from lower to higher degrees—the product of all irreducible factors with degree in $I$ is obtained as $\gcd\left(f, \prod_{i \in I}(x^{q^i} - x)\right)$ and removed from $f$ (Exercise 14.6). If we are lucky, then each of these polynomials is already irreducible; otherwise a fine distinct-degree factorization à la Algorithm 14.3 is performed. Experiments with random polynomials show that on average, this is only necessary for small degrees. In BIPOLAR, the intervals grow linearly in size; this takes into account the fact that random polynomials tend to have many small but few large factors.

Every time a factor is split off, an irreducibility test (similar to Algorithm 14.36) for the remaining polynomial is started up on a second processor. Much of the data required for this test has already been computed in the distinct-degree factorization phase.

Figure 15.6 gives some running times with BIPOLAR from 1998 on two Sun Sparc Ultra 1 computers rated at 167 MHz each; one running the distinct-degree algorithm described above, and the second one performing the irreducibility test in parallel. The timings are the maximum of the CPU times on both machines. An implementation on a parallel machine can factor polynomials in $\mathbb{F}_2[x]$ of degree over one million (Bonorden, von zur Gathen, Gerhard, Müller & Nöcker 2001).

The abort degree is the degree to which the distinct-degree factorization had progressed when the computation was finished. It is printed in bold if the irreducibility test won; otherwise the distinct-degree factorization terminated when

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

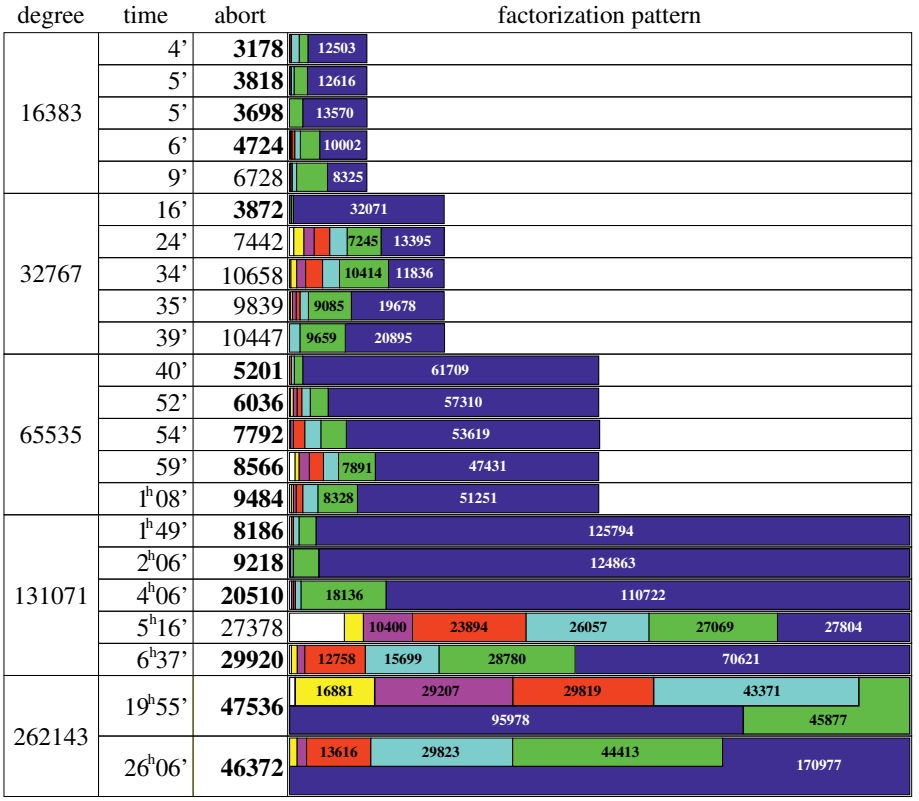



FIGURE 15.6: Factoring polynomials in $\mathbb{F}_2[x]$ with BIPOLAR.

degree about $\max\{m_1/2, m_2\}$ was reached, where $m_1, m_2$ are the degrees of the largest and the second largest factor of the input polynomial, respectively.

For factoring in $\mathbb{F}_p[x]$ for a prime $p$ and in $\mathbb{Q}[x]$, NTL first computes the squarefree decomposition. Over finite prime fields, both the algorithms of Berlekamp (1967,1970) and Shoup (1995) (who also does distinct- and equal-degree factorization) are implemented. For efficiency reasons, there are special factoring subroutines for “small” primes $p$ that fit into one machine word. Figure 15.7 shows some running times in NTL for various primes and degrees. The timings are averages for 10 pseudorandomly chosen inputs.

To factor a squarefree polynomial with integral coefficients in $\mathbb{Q}[x]$, NTL first computes—after extracting its content—its irreducible factorizations modulo several “small” primes, thus (hopefully) gaining some information about the factorization pattern in $\mathbb{Q}[x]$, as described at the end of Section 15.3. Then one of these primes is selected and the factorization is lifted using a variant of Algorithm 15.17. For each product in the factor combination stage, the divisibility test is only executed after checking that the degree of the product is compatible with all modular factorization patterns, and then performing the constant coefficient test.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

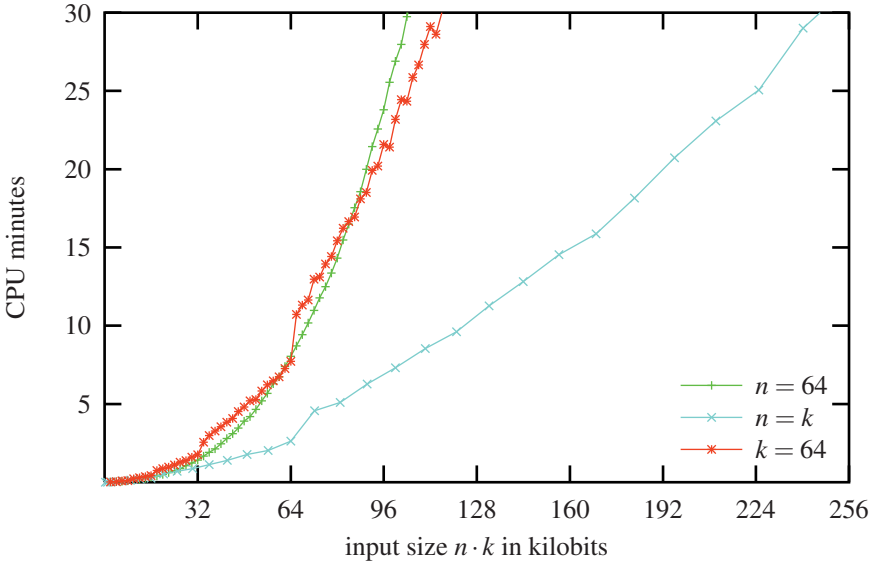



FIGURE 15.7: Factoring polynomials of degree $n-1$ modulo $k$-bit primes in NTL.

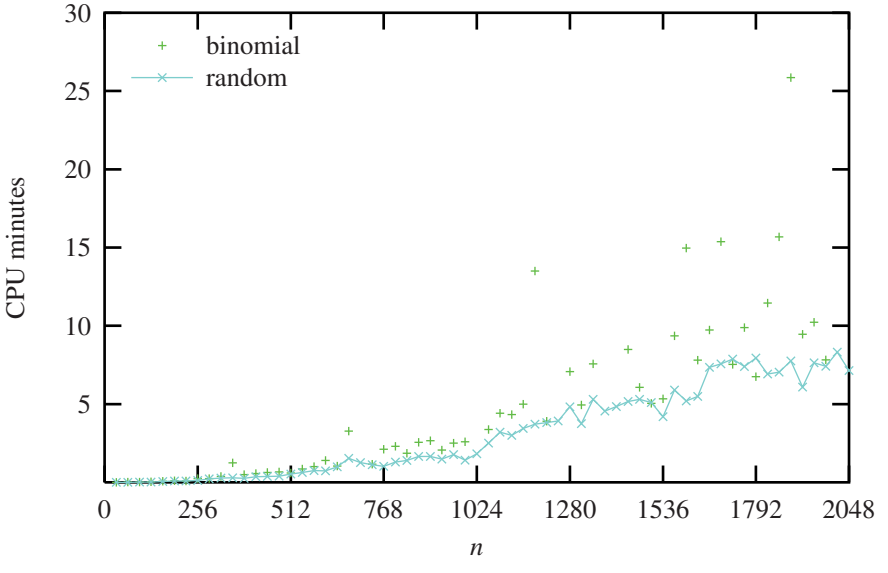



FIGURE 15.8: Factoring $x^{n-1}-1$ in $\mathbb{Z}[x]$ in NTL (green crosses). The timings depend highly on the factorization of $n-1$. The test series is for $n = 32, 64, 96, \ldots, 2048$. The running times for the seven values $n = 704, 1024, 1248, 1376, 1408, 2016$, and 2048 were above four hours, in five of these cases even over one day. For comparison, we have included the average timings for 10 pseudorandom polynomials of degree $n-1$ with coefficients in $\{-1, 0, 1\}$ (blue curve).

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

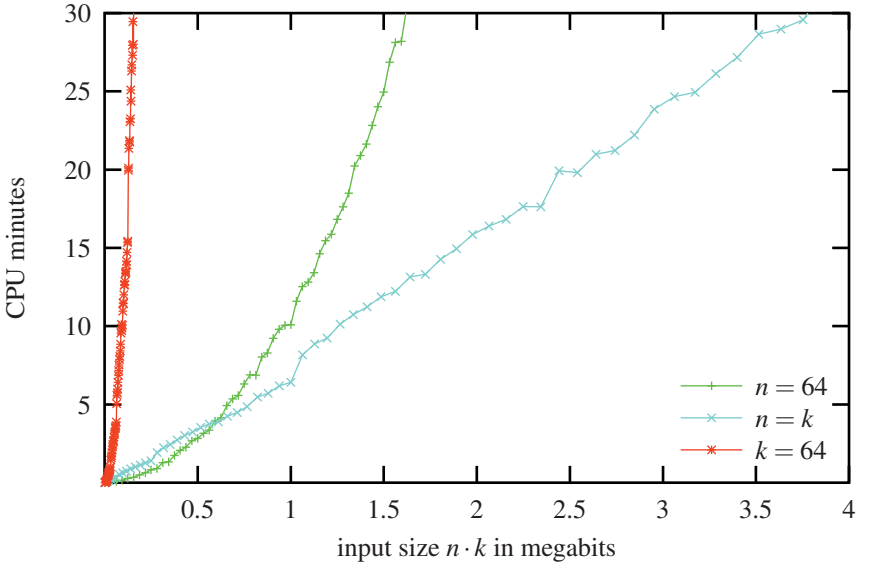



FIGURE 15.9: Factoring products of two pseudorandom polynomials of degree $(n/2) - 1$ with about $k/2$-bit integer coefficients in NTL.

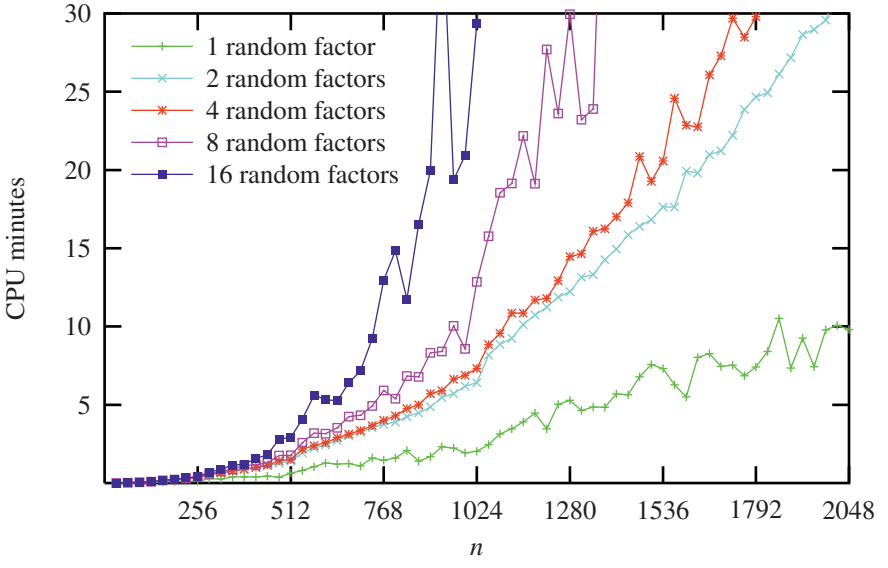



FIGURE 15.10: Factoring polynomials of degree about $n$ with about $n$-bit integer coefficients in NTL. The green curve is for pseudorandom polynomials, the blue curve for products of two pseudorandom polynomials of degree $(n/2) - 1$ with about $n/2$-bit coefficients, and so on.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

Figures 15.8 through 15.10 give some running times in NTL for pseudorandom polynomials with various degrees, coefficient sizes, and factorization patterns. The timings are averaged over 10 pseudorandomly chosen inputs. Since random polynomials are irreducible with high probability (see Notes 15.3), we also took polynomials with a designed number of pseudorandom factors of the same degree as input; in fact, these were the irreducible factors in almost all cases.

Figure 15.10 indicates that the algorithms take longer if there are more factors, as expected. For a fixed input size $n \cdot k$, the "diagonal" case $n = k$ appears to be the most favorable to the implementation, as in Figure 15.9. A comparison to Figure 15.7 indicates that the software factors random polynomials with $k$-bit coefficients over the integers much faster than modulo $k$-bit primes. (A major difference between the two factorization tasks is that random polynomials of degree $n$ are irreducible over $\mathbb{Q}$ with high probability, but modulo a prime only with probability about $1/n$.) Presumably the reason is that the modular factorization stage, which takes about $O^{\sim}(n^2 \log p + n \log^2 p)$ word operations, uses a small prime $p$ with only about $\log_2 k$ bits when factoring over $\mathbb{Z}$ by Hensel lifting.

**Notes.** **15.1.** The first factoring algorithms for $\mathbb{Z}[x]$ are due to von Schubert (1793), Gergonne (1822), and Kronecker (1882, 1883). In our terminology, they use a "small primes approach" with linear moduli $x - u_i$ for small integers $u_i$, plus factor combination. It involves the factorization of large integers and is utterly impractical.

**15.2.** Algorithm 15.2 appears in Musser (1971). Our algorithms become somewhat simpler when the input is monic. One can reduce the general case to this, replacing $f \in \mathbb{Z}[x]$ by $\mathrm{lc}(f)^{n-1} f(x/\mathrm{lc}(f))$, but this is computationally disadvantageous.

In our approach, factors are selected according to the number of irreducible modular factors that they comprise. One might also consider selecting them according to their degree, but Collins (1979) argues that this is disadvantageous. Collins & Encarnación (1996) and Abbott, Shoup & Zimmermann (2000) propose some heuristic techniques for the factor combination stage.

Arjen Lenstra (1984, 1987) gives algorithms for factoring polynomials over algebraic number fields.

**15.3.** The Swinnerton-Dyer polynomials were suggested by H. P. F. Swinnerton-Dyer, as Berlekamp (1970) mentions. Kaltofen, Musser & Saunders (1983) investigate generalizations and also give bounds on the coefficient sizes.

Frobenius (1896) had found his theorem in 1880, and Chebotarev (1926) generalized it in the following way, as already conjectured by Frobenius. In the full symmetric group $S_n$, all permutations with the same cycle structure form a conjugacy class, but this may not be true in other Galois groups. As an example, the two 4-cycles in the Galois group of $f_2$ in Table 15.3 are not conjugate within that group. But still each set of permutations with the same cycle structure is a union of conjugacy classes. While Frobenius' theorem refers to cycle structure (and primes with that factorization pattern), Chebotarev's result proves the corresponding density estimate for the finer division into conjugacy classes (and primes whose Frobenius automorphism lies within that conjugacy class).

Van der Waerden (1934) proved that random integer polynomials have the full symmetric group as their Galois group with probability 1. In particular, they are probably

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

irreducible. See also Dörge (1926). Quantitative estimates are in Gallagher (1973). Wilf (1994), §4.1, proves that the average number of cycles of a random permutation on $n$ letters is $H_n \in \ln n + O(1)$, where $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$ is the $n$th harmonic number (Section 23.2).

**15.4.** Legendre (1785) factors some integer polynomials by a $p$-adic method. One of his examples is in Exercise 15.11. Another example gives two cubic factors of a polynomial of degree six. But he does not state a general method, and cautions the reader (pages 506/507): *Ces méthodes sont fort imparfaites, mais l'utilité de leur objet nous a engagés à les insérer ici, quelque petit que soit le nombre des cas où on peut s'en servir avec succès.*[1]

Hensel (1918) introduced the $p$-adic numbers, and his factoring method *Hensel lifting* was first used in a computer algebra context by Zassenhaus (1969); see also Kempfert (1969). As with so many topics of this book, Gauß had preempted them all. In his Nachlass (Gauß 1863a, 1863b, see page 372) we find in articles 373 and 374 an explicit description of the lifting procedure modulo prime powers, and Gauß concludes *si functio X aequales non habeat divisores secundum modulum p, eam secundum modulum $p^k$ similiter in factores discerpi posse, uti secundum modulum p. At si X divisores aequales habeat, res fit multo magis complicata neque adeo ex principiis praecedentibus prorsus exhauriri potest.*[2] He then even considers the case where multiple factors modulo $p$ may occur, but the calculations in his manuscript end dramatically in the middle of an equation.

Indeed, an integer polynomial $f$ that is squarefree modulo a prime $p$ has a unique factorization modulo any power $p^k$, and this is easily computed by Hensel lifting. But when $f$ is not squarefree modulo $p$, this factorization is quite tricky. Hensel lifting reduces the problem to the case where $f$ is a power of an irreducible polynomial modulo $p$. There may be exponentially (in $\deg f$) many irreducible factors, but a representation of them can still be computed in polynomial time (in $\deg f, \log p$, and $k$) if the discriminant is nonzero modulo $p^k$ (von zur Gathen & Hartlieb 1998). This is based on polynomial-time factoring algorithms over the $p$-adic integers $\mathbb{Z}_{(p)}$; see Chistov (1990) and Cantor & Gordon (2000). Exercise 15.18 shows how factoring the innocuous polynomial $x$ modulo a composite number is a nontrivial task.

**15.5.** Victor Shoup has implemented a somewhat more efficient variant of Algorithm 15.17 in NTL. It is described in the 1999 edition of this book. A different approach to lift factors simultaneously is in von zur Gathen (1984a).

**15.6.** The prime power factorization algorithm 15.19, based on Hensel lifting, is essentially due to Zassenhaus (1969). Loos (1983) gives an algorithm based on Hensel lifting for computing all rational roots of an integer polynomial.

A conceptually simple way to factor bivariate polynomials over small finite fields is to make an extension of prime degree larger than $\deg(f)$; then all factors over the extension are actually in the ground field (von zur Gathen 1985). However, this approach is computationally inferior to the other solutions presented.

Trager (1976) shows that if $F \subseteq E$ is a finite Galois extension of fields, $f \in F[x]$ is irreducible, and $g \in E[x]$ is an irreducible factor of $f$, then the norm of $g$ is a power of $f$.

Moses & Yun (1973) propose the EZ-GCD algorithm for computing gcds in $\mathbb{Z}[x]$ and rings of multivariate polynomials via Hensel lifting; see also Lauer (2000). Yun (1976)

[1] These methods are quite imperfect, but the importance of their goal induced us to insert them here, however small the number of cases may be where they can be used successfully.

[2] If the polynomial $X$ is squarefree modulo $p$, then it factors modulo $p^k$ in the same way as modulo $p$. But if $X$ has multiple factors, the task is much more complicated and cannot even be solved by the preceding principles in a straightforward manner.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

gives a similar algorithm for computing the squarefree decomposition of a multivariate polynomial.

**15.7.** The blocking strategy was introduced by von zur Gathen & Shoup (1992). The average number of irreducible factors of a random polynomial with coefficients in a finite field appears in Berlekamp (1984), exercise 3.6 (already in the 1968 edition).

Figure 15.6 is from von zur Gathen & Gerhard (1996). The running time depends very much on the factorization pattern, not just on the degree. A similar phenomenon happened with our factorizations in $\mathbb{Z}[x]$. Among the ten inputs leading to one data point in Figures 15.8 through 15.10, the standard deviation was, in some cases, about as large as the average value.

In our factoring experiments with products of pseudorandom polynomials in $\mathbb{Z}[x]$, the pseudorandom factors were almost always irreducible; exceptions occurred up to degree 7 with 7-bit coefficients, and also for higher degrees with coefficients in $\{-1,0,1\}$.

## Exercises.

15.1 (i) Prove **Eisenstein's theorem**: If $f \in \mathbb{Z}[x]$ and $p \in \mathbb{N}$ is a prime number such that $p \nmid \operatorname{lc}(f)$, $p$ divides all other coefficients of $f$, and $p^2 \nmid f(0)$, then $f$ is irreducible in $\mathbb{Q}[x]$.

(ii) Conclude that for any $n \in \mathbb{N}$, the polynomial $x^n - p$ is irreducible in $\mathbb{Q}[x]$.

15.2 Trace Algorithm 15.2 on factoring $f = 30x^5 + 39x^4 + 35x^3 + 25x^2 + 9x + 2 \in \mathbb{Z}[x]$. Choose the prime $p = 5003$ in step 2.

15.3 Here are the irreducible factorizations of the monic polynomial $f \in \mathbb{Z}[x]$ of degree 6 modulo some small primes:

$$\begin{aligned} f &= (x+1)^2 \cdot (x^2+x+1) \cdot (x^4+x^3+x^2+x+1) \in \mathbb{F}_2[x] \\ f &= (x+3) \cdot (x^3+3) \cdot (x^4+4x^3+2x^2+x+4) \in \mathbb{F}_7[x] \\ f &= (x+9) \cdot (x^2+2x+4) \cdot (x^5+5) \in \mathbb{F}_{11}[x] \end{aligned}$$

What can you say about the degrees of the irreducible factors of $f$ in $\mathbb{Z}[x]$?

15.4$^{\longrightarrow}$ Compute the coefficients of the Swinnerton-Dyer polynomial

$$f = (x+\sqrt{-1}+\sqrt{2})(x+\sqrt{-1}-\sqrt{2})(x-\sqrt{-1}+\sqrt{2})(x-\sqrt{-1}-\sqrt{2}) \in \mathbb{Z}[x]$$

and its factorizations modulo $p = 2,3,5$. Prove that $f$ is irreducible.

15.5 We have discussed big prime and prime power approaches to factoring in $\mathbb{Z}[x]$. Use Chebotarev's theorem to explain why a *small primes* approach is not promising.

15.6 Construct two polynomials which have factorization patterns modulo $2,3,5$ as in Example 15.7, such that the first one is irreducible in $\mathbb{Z}[x]$ and the second one splits into two irreducible factors of degrees 3 and 5.

15.7$^{*}$ Let $n = p_1 p_2$, where $p_1, p_2 \in \mathbb{N}$ are distinct odd primes. The $n$th cyclotomic polynomial $\Phi_n$ is irreducible in $\mathbb{Z}[x]$. Prove that it splits modulo any prime $p$ into at least two factors. Hint: Lemma 14.50. Show that the number of factors is at least $p_1 - 1$ if $(p_1 - 1) \mid (p_2 - 1)$.

15.8$^{**}$ Let $p \in \mathbb{N}$ be prime. Prove that all irreducible factors modulo $p$ of a Swinnerton-Dyer polynomial have degree at most 2, and if $p$ does not divide the discriminant, they are either all linear or all quadratic.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

15.9** Let $\mathbb{F}_q$ be a finite field with $q$ elements, for an odd prime power $q$, let $x,y$ be indeterminates over $\mathbb{F}_q$, and

$$f = (x+\sqrt{y}+\sqrt{y+1})(x+\sqrt{y}-\sqrt{y+1})(x-\sqrt{y}+\sqrt{y+1})(x-\sqrt{y}-\sqrt{y+1}).$$

Show that $f \in \mathbb{F}_q[x,y]$ and that $f$ is irreducible, but that $f(x,u) \in \mathbb{F}_q[x]$ splits into at least two factors for all $u \in \mathbb{F}_q$.

15.10* A **partition** of a positive integer $n$ is a sequence $\lambda = (\lambda_1, \ldots, \lambda_r)$ of positive integers such that $\lambda_1 \geq \cdots \geq \lambda_r$ and $n = \lambda_1 + \cdots + \lambda_r$; $r$ is the length of the partition. For example, if $F$ is a field and $f \in F[x]$ of degree $n$, then the factorization pattern of $f$, with the degrees of the factors in descending order, is a partition of $n$.

If $\lambda = (\lambda_1, \ldots, \lambda_r)$ and $\mu = (\mu_1, \ldots, \mu_s)$ are two partitions of $n$, then we say that $\lambda$ is finer than $\mu$ and write $\lambda \preccurlyeq \mu$ if there is a surjective map $\sigma\colon \{1,\ldots,r\} \longrightarrow \{1,\ldots,s\}$ such that $\mu_i = \sum_{\sigma(j)=i} \lambda_j$ for all $i \leq s$. For example, $\lambda = (4,2,1,1)$ is finer than $\mu = (5,3)$, as furnished by $\sigma(1) = \sigma(3) = 1$ and $\sigma(2) = \sigma(4) = 2$. (The function $\sigma$ need not be unique.) In particular, $(n)$ is the coarsest and $(1,1,\ldots,1)$ is the finest partition of $n$.

(i) Prove that if $f \in \mathbb{Z}[x]$ has degree $n$ and $p \in \mathbb{N}$ is prime, $\mu$ is the factorization pattern of $f$ in $\mathbb{Q}[x]$, and $\lambda$ is the factorization pattern of $f$ mod $p$ in $\mathbb{F}_p[x]$, then $\mu \succcurlyeq \lambda$.

(ii) Show that $\lambda \preccurlyeq \lambda$, $\lambda \preccurlyeq \mu \Longrightarrow \neg(\mu \preccurlyeq \lambda)$, and $\lambda \preccurlyeq \mu \preccurlyeq \nu \Longrightarrow \lambda \preccurlyeq \nu$ holds for all partitions $\lambda, \mu, \nu$ of $n$, so that $\preccurlyeq$ is a partial order on the set of all partitions of $n$.

(iii) Enumerate all partitions of $n = 8$, and draw them in form of a directed graph, with an edge from $\lambda$ to $\mu$ if $\mu$ is a direct successor of $\lambda$ with respect to the order $\preccurlyeq$, so that $\lambda \preccurlyeq \mu$, $\lambda \neq \mu$, and $\lambda \preccurlyeq \nu \preccurlyeq \mu \Longrightarrow \lambda = \nu$ or $\nu = \mu$ for all partitions $\nu$.

(iv) Use (iii) to show that there exist partitions $\lambda, \mu$ of 8 that do not have a supremum with respect to $\preccurlyeq$. (Thus the partitions do not form a "lattice" in the sense of order theory, not to be confused with the $\mathbb{Z}$-module lattices in Chapter 16.)

(v) Let $a_{n,r}$ denote the number of partitions of $n$ of length $r$. Thus $a_{n,1} = a_{n,n} = 1$ and $a_{n,r} = 0$ for $1 \leq n < r$. Prove the recursion formula $a_{n,r} = \sum_{1 \leq j \leq r} a_{n-r,j}$ for $1 \leq r < n$. Calculate $a_{n,r}$ for $1 \leq r \leq n \leq 8$, and compare with your results from (iii).

15.11 (Legendre 1785, p. 490) Let $f = x^3 - 292x^2 - 2\,170\,221x + 6\,656\,000 \in \mathbb{Z}[x]$. Find 13-adic linear factors $x - a_i$ with $f \operatorname{rem} x - a_i \equiv 0 \bmod 13^{2^i}$ for $i = 0,1,2$, starting with $a_0 = 0$.

15.12 Prove Lemma 15.9 (ii).

15.13 Suppose that the monic polynomial $f \in \mathbb{Z}[x]$ has degree 8, and $p$ is a prime so that $f$ mod $p = g_1 g_2 g_3$ factors into three irreducible and pairwise coprime polynomials $g_1, g_2, g_3 \in \mathbb{F}_p[x]$ with $\deg g_1 = 1$, $\deg g_2 = 2$, and $\deg g_3 = 5$.

(i) What can you say about the possible factorizations of $f$ modulo $p^{100}$?

(ii) What can you say about the possible factorizations of $f$ in $\mathbb{Q}[x]$?

(iii) Suppose $q$ is another prime for which $f$ mod $q = h_1 h_2$ with $h_1, h_2 \in \mathbb{F}_q[x]$ irreducible and $\deg h_1 = \deg h_2 = 4$. What can you say about the possible factorizations of $f$ in $\mathbb{Q}[x]$, using all this information?

15.14 Let $f = x^{15} - 1 \in \mathbb{Z}[x]$. Take a nontrivial factorization $f \equiv gh \bmod 2$ with $g, h \in \mathbb{Z}[x]$ monic and of degree at least 2. Compute $g^*, h^* \in \mathbb{Z}[x]$ such that

$$f \equiv g^* h^* \bmod 16, \quad \deg g^* = \deg g, \quad g^* \equiv g \bmod 2.$$

Show your intermediate results. Can you guess some factors of $f$ in $\mathbb{Z}[x]$?

15.15 Let $f = 14x^4 + 15x^3 + 42x^2 + 3x + 1 \in \mathbb{Z}[x]$.

(i) Find a suitable prime $p \in \mathbb{N}$ such that $f$ mod $p$ is squarefree and has degree 4.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

(ii) Compute the irreducible factorization of $f \bmod p$ in $\mathbb{F}_p[x]$. Choose two factors $g, h \in \mathbb{Z}[x]$ that are coprime modulo $p$ such that $h$ is monic and irreducible modulo $p$ and $f \equiv gh \bmod p$. Determine $s, t \in \mathbb{Z}[x]$ with $sg + th \equiv 1 \bmod p$.

(iii) Execute two successive Hensel steps (Algorithm 15.10 for $m = p$ and $m = p^2$) to obtain a factorization $f \equiv g^* h^* \bmod p^4$ with $g \equiv g^* \bmod p$ and $h \equiv h^* \bmod p$. Can you derive a factorization of $f$ in $\mathbb{Q}[x]$ from it?

15.16⟶ Consider the polynomial

$$\begin{aligned} f = {} & x^5 + \left(3y^3 + 39y^2 + 50y + 28\right)x^4 + \left(36y^5 + 2y^4 + 47y^3 + 63y^2 + 49y + 58\right)x^3 \\ & + \left(91y^6 + 18y^5 + 81y^4 + 37y^3 + 36y^2 + 53y + 64\right)x^2 \\ & + \left(74y^7 + 54y^6 + 24y^5 + 39y^4 + 71y^3 + 18y^2 + 93y + 53\right)x \\ & + \left(62y^6 + 72y^5 + 87y^4 + 27y^3 + 19y^2 + 61y\right) \in \mathbb{F}_{97}[x, y]. \end{aligned}$$

(i) Compute a factorization $f \equiv gh \bmod y$ with coprime nonconstant polynomials $g, h \in \mathbb{F}_{97}[x]$, and polynomials $s, t \in \mathbb{F}_{97}[x]$ with $sg + th = 1$.

(ii) Execute two successive Hensel steps (Algorithm 15.10 with $m = y$ and $m = y^2$) to obtain polynomials $g^*, h^* \in \mathbb{F}_{97}[x, y]$ such that $f \equiv gh \bmod y^4$, $g^* \equiv g \bmod y$, and $h^* \equiv h \bmod y$.

15.17 Complete the proof of Theorem 15.11.

15.18 (Shamir 1993) Let $N = p \cdot q$ be the product of two distinct primes $p, q$.

(i) Show that $u = p^2 + q^2$ is a unit in $\mathbb{Z}_N^\times$.

(ii) Verify the factorization $x \equiv u^{-1}(px + q)(qx + p) \bmod N$.

(iii) Prove that the two linear factors in (ii) are irreducible in $\mathbb{Z}_N[x]$. Hint: CRT.

15.19* Let $N = p_1 \cdots p_s$ be a product of $s$ distinct primes, and $f \in \mathbb{Z}_N[x]$ be monic and squarefree.

(i) Let $g_1 \in \mathbb{Z}_{p_1}[x]$ be irreducible, and $g \in \mathbb{Z}_N[x]$ with $g \equiv g_1 \bmod p_1$ and $g \equiv 1 \bmod p_i$ for $i \geq 2$. Prove that $g$ is irreducible in $\mathbb{Z}_N[x]$.

(ii) Assume that we have factored $f$ modulo each $p_i$. Determine the factorization of $f$ into irreducible polynomials in $\mathbb{Z}_N[x]$. How many irreducible factors are there, in terms of the numbers of irreducible factors modulo each $p_i$?

(iii) How many irreducible factors does $x^3 - x$ have modulo 105? Find four of them.

15.20 This exercise discusses a variant of Hensel lifting with linear convergence.

(i) One step works as follows. In Algorithm 15.10, we have an additional input $p \in R$, and the congruence $sg + th \equiv 1$ should hold modulo $p$ instead of $m$. In step 1, we perform all computations modulo $mp$ instead of $m^2$, and step 2 is omitted completely. Prove that then the output specifications of Algorithm 15.10 for $f, g^*, h^*$ hold if $m^2$ is replaced by $mp$.

(ii) Now we start with a factorization of $f$, including the polynomials $s, t$, as specified in Algorithm 15.10 for $m = p$, and want to compute a factorization modulo $p^l$ for some $l \in \mathbb{N}$. Show that for $R = \mathbb{Z}$, this takes $O(\mathsf{M}(n)\mathsf{M}(l \log p) \log l)$ word operations when using the quadratic lifting algorithm 15.10 for $m = p, p^2, p^4, p^8, \ldots$, and $O(\mathsf{M}(n)\mathsf{M}(l \log p)l)$ word operations when using the linear lifting algorithm from (i) for $m = p, p^2, p^3, p^4, \ldots$.

In fact, by employing fast lazy multiplication techniques (van der Hoeven 1997), the cost for linear Hensel lifting can be reduced to $O^\sim(nl \log p)$ as well (Bernardin 1998, private communication).

15.21 Let $R$ be a ring (commutative, with 1) and $f \in R[x], g = \sum_{0 \leq i \leq m} g_i x^i, h = \sum_{0 \leq i \leq k} h_i x^i$ be polynomials such that $n = \deg f = m + k \geq 1$ and $\mathrm{lc}(f) = g_m h_k$. Regarding the $n$ coefficients $g_0, \ldots, g_{m-1}$, $h_0, \ldots, h_{k-1}$ as indeterminates, we define $n$ polynomials $\varphi_0, \ldots, \varphi_{n-1}$ in these indeterminates by letting $\varphi_i$ be the coefficient of $x^i$ in $gh - f$ for $0 \leq i < n$.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

(i) Prove that the Jacobian $J \in R[g_0,\ldots,g_{m-1},h_0,\ldots,h_{n-1}]^{n\times n}$, whose $i$th row comprises the partial derivatives $\partial\varphi_i/\partial h_j$ and $\partial\varphi_i/\partial g_j$, is precisely the Sylvester matrix of $g$ and $h$.

(ii) Conclude that for specific values of the coefficients of $g,h$ and a given $p \in R$ such that $\operatorname{lc}(f)$ is a unit modulo $p$, there exist $s,t \in R[x]$ such that $sg+th \equiv 1 \bmod p$ if and only if $J$ is invertible modulo $p$. Hint: Exercise 6.15.

15.22 Prove the running time estimate of Theorem 15.18 for the case $R = F[y]$.

15.23 Let $f = 6x^5+23x^4+51x^3+65x^2+65x+42 \in \mathbb{Z}[x]$ and $p = 11$.

(i) Compute the irreducible factorization of $f$ mod $p$.

(ii) Use Algorithm 15.17 to lift the factorization above to a factorization of $f$ modulo $p^4$.

(iii) Try to find nontrivial factors of $f$ in $\mathbb{Z}[x]$ via factor combination.

15.24* Prove Theorem 15.23.

15.25* (i) Show that the bivariate polynomial factorization algorithm 15.24 works correctly when $F$ is a field of characteristic zero. First convince yourself that $u = v$ is the squarefree part of $g$.

(ii) Let $\mathbb{F}_q$ be a finite field of characteristic $p$ and $h \in \mathbb{F}_q[x,y]$. Prove that $h$ is a $p$th power if and only if $\partial h/\partial x = \partial h/\partial y = 0$.

(iii) Show that $h$ is squarefree if it is coprime to one of $\partial h/\partial x$ or $\partial h/\partial y$. Hint: Exercise 14.22.

(iv) Prove that in step 3 of Algorithm 15.24, $\gcd(u,\partial u/\partial x) = \gcd(w,\partial w/\partial x) = 1$ in $\mathbb{F}_q(y)[x]$ and $\gcd(v,\partial v/\partial y) = 1$ in $\mathbb{F}_q(x)[y]$, and that $u,v$, and $w$ are squarefree.

(v) Now assume that $h$ is an irreducible factor of $g$ with multiplicity $e$ in step 3 of Algorithm 15.24. Conclude from the above that if $p \nmid e$, then $h \mid vw$ in step 3 and $h \nmid g$ in step 8, and that $h^e$ still divides $g$ in step 8 if $p \mid e$.

(vi) Prove that Algorithm 15.24 works correctly when $F = \mathbb{F}_q$.

15.26* (Gerhard 2001a) This exercise discusses a small primes modular algorithm for computing the squarefree decomposition of a primitive polynomial $f \in \mathbb{Z}[x]$.

(i) Let $R$ be a UFD and $f \in R[x]$ nonconstant and primitive. Prove that there exist primitive squarefree and pairwise coprime polynomials $g_1,\ldots,g_m \in R[x]$ such that $g_m$ is nonconstant and $f = g_1g_2^2\cdots g_m^m$. Show that $m$ is unique and the $g_i$ are unique up to multiplication by units in $R^\times$.

For the special cases $R = \mathbb{Z}$ or $R = F[y]$, where $F$ is a field, we can make the decomposition above unique by stipulating that $\operatorname{lc}(g_i) \in R$ be positive or monic, respectively, assuming that $\operatorname{lc}(f)$ is also positive or monic. Then we call the sequence $(g_1,\ldots,g_m)$ the **primitive squarefree decomposition** of $f$.

(ii) Now let $R = \mathbb{Z}$, $p$ be a prime not dividing $\operatorname{lc}(f)$ and $f \equiv \operatorname{lc}(f)h_1h_2^2\cdots h_k^k \bmod p$ be the squarefree decomposition of $f$ modulo $p$, with monic $h_1,\ldots,h_k \in \mathbb{Z}[x]$ that are squarefree and pairwise coprime modulo $p$, and $h_k \neq 1$. Show that $k \geq m$ and the modular squarefree part $h_1\cdots h_k$ divides modulo $p$ the squarefree part $g = g_1\cdots g_m$ of $f$. Prove that $k = m$ and $g_i \equiv \operatorname{lc}(g_i)h_i \bmod p$ for all $i$ if $p$ does not divide the (nonzero) discriminant $\operatorname{res}(g,g') \in \mathbb{Z}$ of $g$.

(iii) Prove the following generalization of Mignotte's bound (Corollary 6.33): If $f,g_1,\ldots,g_m \in \mathbb{Z}[x]$ are nonzero polynomials with $f = g_1\cdots g_m$ and $n = \deg f$, then

$$\prod_{1\leq i\leq m} \|g_i\|_1 \leq (n+1)^{1/2}2^n\|f\|_\infty.$$

(iv) Design a small primes modular algorithm for computing the primitive squarefree decomposition of $f$, in analogy to the small primes modular gcd algorithm 6.38. Your algorithm should check that the result is correct, so that it is Las Vegas, and use $O^\sim(n^2+n\log A)$ word operations if $A = \|f\|_\infty$.

15.27* Using Exercise 15.26, design a prime power modular algorithm for computing the squarefree decomposition of a primitive polynomial $f \in \mathbb{Z}[x]$. Your algorithm should check that the result is correct, so that it is Las Vegas, and use $O^\sim(n^2+n\log A)$ word operations if $n = \deg f$ and $A = \|f\|_\infty$.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

15.28 We have indicated in Section 15.7 that factoring a random polynomial with $k$-bit coefficients over the integers is computationally faster than factoring the same polynomial modulo a $k$-bit prime. This suggests a factoring algorithm over finite prime fields which first factors the input polynomial over the integers and then calls the factoring algorithm over finite fields for its (in $\mathbb{Z}[x]$) irreducible factors. Explain why this is not of much help for random polynomials.

15.29 Let $R$ be a ring, $1 \leq k < r$, $f_1, \ldots, f_r \in R[x]$ monic, nonconstant, and pairwise Bézout-coprime, $b \in R^\times$, $g = bf_1 \cdots f_k$, and $h = f_{k+1} \cdots f_r$. Show that $g$ and $h$ are Bézout-coprime. (Hint: Proceed as in the solution of Exercise 10.13.) More precisely, prove that there exist polynomials $s, t \in R[x]$ such that $sg + th = 1$, $\deg s < \deg h$, and $\deg t < \deg g$.

15.30 The aim of this exercise is to shave off the factor $\log r$ in the running time estimate of Theorem 15.18 when using classical arithmetic. In addition to the input specification of Algorithm 15.17, we assume that the $f_i$ are sorted by degree, so that $n_1 = \deg f_1 \leq n_2 = \deg f_2 \leq \cdots \leq n_r = \deg f_r$. Let $l_i = \lfloor \log n_i \rfloor$ for all $i$ and $e = e(n_1, \ldots, n_r) = \lceil \log \sum_{1 \leq i \leq r} 2^{l_i} \rceil$. (As usual, log denotes the binary logarithm.)

(i) Assume $r \geq 2$ and let $1 \leq k < r$ be maximal such that $\sum_{k < i \leq r} 2^{l_i} \leq 2^{e-1}$. Prove that such a $k$ exists, that actually equality holds, and conclude that $e(n_1, \ldots, n_k) \leq e-1$ and $e(n_{k+1}, \ldots, n_r) \leq e-1$.

(ii) We replace the definition of $k$ in step 2 of Algorithm 15.17 by the definition as in (i) and denote by $T(n_1, \ldots, n_r)$ the cost of the algorithm with classical arithmetic. Prove that there is a positive constant $c$ such that

$$T(n) \leq c 2^e l^2 \mu^2, \qquad T(n_1, \ldots, n_r) \leq T(n_1, \cdots, n_k) + T(n_{k+1}, \ldots, n_r) + c 2^{2e} l^2 \mu^2,$$

with $\mu$ as in Theorem 15.18, $n = n_1 + \cdots + n_r$, and $e = e(n_1, \ldots, n_r)$. Conclude that $T(n_1, \ldots, n_r) \leq \frac{4}{3} c 2^{2e} l^2 \mu^2 \in O(n^2 l^2 \mu^2)$.

15.31* You are to design a small primes modular algorithm computing the squarefree decomposition of a bivariate polynomial. For the sake of simplicity, we let $F$ be a field of characteristic zero and $f \in F[x,y]$ be primitive with respect to $x$ with $n = \deg f > 0$ and $\mathrm{lc}_x(f) \in F[y]$ monic.

(i) Let $u \in F$ be such that $\deg_x f(x,u) = \deg_x f$, let $f = g_1 g_2^2 \cdots g_m^m$ be the primitive squarefree decomposition of $f$ with respect to $x$, as defined in Exercise 15.26 (i), with $g_1, \ldots, g_m \in F[x,y]$ that are squarefree, pairwise coprime, and primitive with respect to $x$ and have monic leading coefficients, and $\deg_x g_m > 0$. Moreover, let $h_1, \ldots, h_k \in F[x]$ be the squarefree decomposition of $f(x,u)/\mathrm{lc}_x(f(x,u))$. Prove that $k = m$ and $g_i(x,u) = \mathrm{lc}_x(g_i)(u) \cdot h_i$ for $1 \leq i \leq m$ if $u$ is not a root of the (nonzero) discriminant $\mathrm{res}_x(f, \partial f/\partial x) \in F[y]$ of $f$ with respect to $x$. Hint: Proceed as in Exercise 15.26.

(ii) Design a small primes modular algorithm for computing the primitive squarefree decomposition of $f$, in analogy to the small primes modular gcd algorithm 6.36. Your algorithm should check that the result is correct, so that it is of Las Vegas type, and use $O^\sim(nd)$ word operations if $\deg_y f = d$.

## Research problem.

15.32 Let $f \in \mathbb{Z}[x]$, $p$ be a prime, and $k \in \mathbb{N}$. Can you find all factorizations of $f$ into irreducible factors modulo $p^k$ in time polynomial in $\deg f$ and $k \log p$? An apparently difficult case is when the discriminant $\mathrm{res}(f, f')$ is zero.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press

La clarté est, en effet, d'autant plus nécessaire,
qu'on a dessein d'entraîner le lecteur plus loin
des routes battues et dans des contrées plus arides.[1]

*Joseph Liouville (1846)*

Mathematics was orderly and it made sense. The answers were
always there if you worked carefully enough, or that's what she said.

*Sue Grafton (1982)*

A good idea, but not as simple in life as it is in theory.

*Philip Friedman (1992)*

[1] Clarity is all the more necessary when one intends to guide the reader further away from the beaten track and into more arid countryside.

https://doi.org/10.1017/CBO9781139856065.019 Published online by Cambridge University Press